

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



MACHINE LEARNING (CO3117)

Report:

Assignment 2

Team LHPD2

Semester 2, Academic Year 2024 - 2025

Teacher:	Nguyen An Khuong	
Students:	Nguyen Quang Phu	- 2252621 (Leader)
	Nguyen Thanh Dat	- 2252145 (Member)
	Pham Huynh Bao Dai	- 2252139 (Member)
	Nguyen Tien Hung	- 2252280 (Member)
	Nguyen Thien Loc	- 2252460 (Member)

HO CHI MINH CITY, MAY 2025

Contents

<i>List of Figures</i>	7
<i>List of Tables</i>	9
1 Abstract	11
2 Description	12
3 Project Organization and Repository Structure	13
3.1 Team Members and Workloads	13
3.2 Project Organization and Requirements	14
3.2.1 Team Collaboration and Version Control	14
3.2.2 GitHub Repository Structure	14
3.2.3 Key Requirements	15
3.3 Repository Structure	15
3.3.1 How to Run This Project	16
3.3.1.1 Clone the Repository	16
3.3.1.2 Environment Setup	16
3.3.2 Project Exploration	16
4 Code Engineering	17
4.1 Datasets	17
4.1.1 sentiment140-dataset	17
4.1.1.1 Dataset Description	17
4.1.1.2 Dataset Format	17
4.1.1.3 Dataset Notes	18

4.1.2	twitter-tweets-sentiment-dataset	18
4.1.2.1	Dataset Description	18
4.1.2.2	Dataset Format	18
4.1.2.3	Dataset Notes	19
4.1.3	twitter-sentiments-dataset	19
4.1.3.1	Dataset Description	19
4.1.3.2	Dataset Format	20
4.1.3.3	Dataset Notes	20
4.2	Data Preprocessing	21
4.2.1	Data Collection Process	21
4.2.2	Data Preprocessing and Merging	21
4.2.3	Data Cleaning and Preparation	22
4.2.4	Exploratory Data Analysis (EDA)	24
4.2.5	Publishing the Merged Dataset on Kaggle	26
4.2.6	Final Remarks	26
4.3	Visualization	27
4.3.1	Relationships of the Attributes	27
4.3.2	Top Words Visualizations	28
4.3.3	The distribution of Text Length	29
4.3.4	Word Frequencies by Labels	31
5	Analyze the Training Process of Models	33
5.1	Building Features	33
5.1.1	Overview	33
5.1.2	Key Components and Functionalities	35
5.1.3	Usage	35
5.2	General Training Methods	35
5.3	Project Workflow and Implementation	36
5.3.1	Training and Evaluation Workflow	37
5.3.2	Implementation Quality and Code Efficiency	37
5.3.3	Use Case Scenarios	37
5.3.4	Data Preprocessing and Model Tuning	38

5.3.5	Performance Analysis and Model Evaluation	38
5.3.6	Documentation and Reproducibility	39
5.3.7	Project Management and Collaboration	39
5.4	Model: Hidden Markov Model	40
5.4.1	Introduction	40
5.4.2	Training Configuration	40
5.4.3	Training and Evaluation Results	41
5.4.4	Performance Analysis	42
5.4.5	Conclusion	43
5.5	Model: BayesNet	43
5.5.1	Introduction	43
5.5.2	Training Configuration	43
5.5.3	Training and Evaluation Results	45
5.5.4	Performance Analysis	45
5.5.5	Conclusion	46
5.6	Model: Support Vector Machine (SVM)	46
5.6.1	Introduction	46
5.6.2	Training Configuration	47
5.6.3	Training and Evaluation Results	47
5.6.4	Performance Analysis	48
5.6.5	Visualization of Training Results	49
5.6.6	Conclusion	50
5.7	Model: Linear Discriminant Analysis (LDA)	51
5.7.1	Introduction	51
5.7.2	Training Configuration	51
5.7.3	Training and Evaluation Results	52
5.7.4	Performance Analysis	53
5.7.5	Visualization of Training Results	53
5.7.6	Conclusion	54
5.8	Model: Logistic Regression	54
5.8.1	Introduction	54
5.8.2	Training Configuration	55

5.8.3	Training and Evaluation Results	55
5.8.4	Performance Analysis	56
5.8.5	Visualization of Training Results	57
5.8.6	Conclusion	58
5.9	Model: XGB	59
5.9.1	Introduction	59
5.9.2	Training Configuration	59
5.9.3	Training and Evaluation Results	59
5.9.4	Performance Analysis	61
5.9.5	Visualization of Training Results	61
5.9.6	Conclusion	64
5.10	Model: Random Forest	64
5.10.1	Introduction	64
5.10.2	Training Configuration	64
5.10.3	Training and Evaluation Results	64
5.10.4	Performance Analysis	66
5.10.5	Visualization of Training Results	66
5.10.6	Conclusion	68
5.11	Model: BiLSTM with CRF	68
5.11.1	Theory of BiLSTM and CRF	68
5.11.2	Introduction	68
5.11.3	Training Configuration	69
5.11.4	Training and Evaluation Results	69
5.11.5	Performance Analysis	71
5.11.6	Conclusion	71
5.12	Feature Selection	71
5.12.1	Chi-Squared - Feature Selection Method	71
5.12.1.1	Feature Extraction and Selection	71
5.12.1.2	Model Training Procedure	72
5.12.1.3	Experimental Setup	72
5.12.1.4	Evaluation Metrics	73
5.12.1.5	Discussion	73

5.12.2	Topic Modeling - Feature Selection Method	74
5.12.2.1	Feature Extraction and Selection	74
5.12.2.2	Model Training Procedure	75
5.12.2.3	Experimental Setup	75
5.12.2.4	Evaluation Metrics	76
5.12.2.5	Discussion	76
5.12.3	Variance - Feature Selection Method	77
5.12.3.1	Feature Extraction and Selection	77
5.12.3.2	Model Training Procedure	77
5.12.3.3	Experimental Setup	78
5.12.3.4	Evaluation Metrics	78
5.12.3.5	Discussion	79
5.13	Dimensionality Reduction with PCA	79
5.13.1	Introduction	79
5.13.2	Implementation	80
5.13.3	Comparative Results	81
5.13.4	Model Learning Curves	83
5.13.5	Best Model Performance	84
5.13.6	Comparison with Original Models	86
5.13.7	Conclusion	86
5.14	Model: Stacking	87
5.14.1	Theory of Stacking	87
5.14.2	Implementation	87
5.14.3	Model 1: Stacking Multiple Logistic Regression	89
5.14.4	Model 2: Stacking Selected Models	89
5.14.5	Conclusion	91
5.15	Model: Voting	91
5.15.1	Theory of Voting	91
5.15.2	Implementation	92
5.15.3	Model 1: Voting Multiple Logistic Regression	93
5.15.4	Model 2: Voting Selected Models	94
5.15.5	Conclusion	96

6	Discussion and Analysis	97
6.1	Reproducibility Implementation	97
6.1.1	Environment Configuration Management	97
6.1.2	Versioned Model Storage	97
6.1.3	Deterministic Execution Control	98
6.1.4	Result Reproducibility	99
6.1.5	Pipeline Automation Architecture	101
6.1.5.1	Training Pipeline	101
6.1.5.2	Prediction Pipeline	102
6.1.5.3	Feature Building Pipeline	103
6.2	Model Comparison for Sentiment Analysis	105
6.2.1	Introduction	105
6.2.1.1	Model Training and Evaluation Workflow	105
6.2.1.2	Evaluation Metrics Overview	106
6.2.2	Baseline Model Comparison (Best-performing Feature Methods)	106
6.2.3	Discussion on Feature Selection and Dimensionality Reduction Methods	107
6.2.3.1	Chi-Squared Feature Selection	107
6.2.3.2	Topic Modeling Feature Selection	108
6.2.3.3	Variance Threshold Feature Selection	109
6.2.3.4	PCA (Principal Component Analysis)	109
6.2.3.5	Raw TF-IDF and CountVectorizer (Without PCA)	110
6.2.3.6	GloVe Embedding Features	111
6.2.4	Comparison with Theory	111
6.2.5	Limitation Discussion	112
6.2.6	Use-case Fit Analysis	112
7	Self-Reflection	113
7.1	Future Developments	113
7.2	Lessons	114
7.3	Special Thanks	115

List of Figures

3.1	Github Repository Structure	15
4.1	Distribution of Target Variable	24
4.2	Overall Text Cleaned Length Distribution	25
4.3	Distribution of Raw Text Lengths	25
4.4	Pairwise Relationships	27
4.5	Top 20 of entire dataset	28
4.6	Top 20 of class Positive	28
4.7	Top 20 of class Negative	29
4.10	Word Frequency by Sentiment	31
4.11	Comparision of Word Frequency	32
5.1	Comparison of Loss Curves for SVM across Different Feature Extraction Methods	49
5.2	Comparison of Training Performance Metrics for SVM across Different Feature Extraction Methods	50
5.3	Loss Curve - LDA with Count Vectorizer	53
5.4	Performance Metrics - LDA with Count Vectorizer	54
5.5	Comparison of Loss Curves for Logistic Regression across Different Feature Extraction Methods	57
5.6	Comparison of Training Performance Metrics for Logistic Regression across Different Feature Extraction Methods	58
5.7	Loss Curves for XGBoost across Different Feature Extraction Methods	62
5.8	Comparison of Training Performance Metrics for XGBoost across Different Feature Extraction Methods	63
5.9	Comparison of Loss Curves for Random Forest across Different Feature Extraction Methods	66

5.10 Comparison of Training Performance Metrics for Random Forest across Different Feature Extraction Methods	67
5.11 Learning Curves for Classification Models (Part 1)	83
5.11 Learning Curves for Classification Models (Part 2)	84
5.12 Best Performance Metrics for Each Classification Model (Count Vectorizer and TF-IDF Methods)	85

List of Tables

5.1	Testing Performance Metrics for Hidden Markov Model	42
5.2	Testing Performance Metrics for Bayesian Network	45
5.3	Training Performance Metrics for SVM	47
5.4	Testing Performance Metrics for SVM	47
5.5	Training Performance Metrics for LDA	52
5.6	Testing Performance Metrics for LDA	52
5.7	Training Performance Metrics for Logistic Regression	55
5.8	Testing Performance Metrics for Logistic Regression	56
5.9	Training Performance Metrics for XGBoost	60
5.10	Testing Performance Metrics for XGBoost	60
5.11	Training Performance Metrics for Random Forest (Cross-Validation)	65
5.12	Testing Performance Metrics for Random Forest	65
5.13	Testing Performance Metrics for BiLSTM with CRF	69
5.14	Testing Classification Report for BiLSTM with CRF	70
5.15	Performance comparison across different models and feature extraction methods.	73
5.16	Performance comparison across different models and feature extraction methods with Topic Modeling Feature Selection.	76
5.17	Model performances using GloVe feature extraction and feature selection . .	79
5.18	Training Performance Metrics (With PCA)	81
5.19	Testing Performance Metrics (With PCA)	81
5.20	Training Performance Metrics (Count & TF-IDF without PCA)	82
5.21	Testing Performance Metrics (Count & TF-IDF without PCA)	82
5.22	Training and Testing Performance of Stacking Multiple Logistic Regression .	89
5.23	Testing Classification Report for Stacking Multiple Logistic Regression . . .	89

5.24	Training and Testing Performance of Stacking Selected Models	91
5.25	Testing Classification Report for Stacking Selected Models	91
5.26	Training and Testing Performance of Voting Multiple Logistic Regression . .	94
5.27	Testing Classification Report for Voting Multiple Logistic Regression	94
5.28	Training and Testing Performance of Voting Selected Models	95
5.29	Testing Classification Report for Voting Selected Models	96
6.1	Model Comparison for Sentiment Analysis (Best-performing features applied)	107

Chapter 1

Abstract

With the increasing adoption of Natural Language Processing (NLP) in various domains, sentiment analysis has become a crucial task in understanding opinions, emotions, and attitudes expressed in text. The ability to automatically classify sentiments in text is highly valuable for applications in social media monitoring, product reviews, and customer feedback analysis. This project aims to develop a Machine Learning (ML) model capable of performing sentiment analysis, distinguishing between different sentiment classes such as positive, negative, and neutral. Our goal is to explore various techniques in sentiment classification, evaluate model effectiveness, and contribute to advancements in automated sentiment analysis.

Chapter 2

Description

In the digital age, people share emotions and opinions through social media, product reviews, and forums. Sentiment analysis, or opinion mining, is a crucial NLP technique for businesses, researchers, and policymakers to analyze public sentiment. However, challenges like sarcasm, ambiguity, and varied linguistic expressions make accurate classification difficult.

For this project, our team (**LHPD2**) will develop a Machine Learning model for sentiment analysis as part of **Assignment 2** in this **Machine Learning** course. Our goal is to classify text into sentiment categories using NLP techniques like word embeddings, recurrent neural networks, and transformer-based models. This involves feature extraction, evaluating classification algorithms, and analyzing model performance.

Sentiment analysis plays a growing role in applications such as customer experience improvement and social media trend detection. However, ethical concerns, including bias in training data and misinterpretation of sentiments, must be addressed. Alongside implementing sentiment classification models, we will explore ways to enhance model fairness and accuracy.

This assignment focuses on applying **engineering techniques** to sentiment analysis using MODELS. Key tasks include **feature transformation, handling high-dimensional data, network architecture design, hyperparameter tuning, and model evaluation**. Additionally, we will apply **feature selection, optimization strategies, and probability modeling** to improve performance, aligning with **data preprocessing, model tuning, and performance analysis**.

Our objective is to gain hands-on experience by focusing on **structured implementation, tuning, and evaluation**, rather than theoretical innovations. This project will emphasize **efficient engineering solutions** to improve sentiment classification across multiple models.

Chapter 3

Project Organization and Repository Structure

3.1 Team Members and Workloads

The project is developed by **Group LHPD2**, consisting of the following members:

No.	Full Name	Student ID	Task Assigned
1	Nguyen Quang Phu	2252621	Team leader; Manage repository and schedule; Stacking & Voting with Logistic Regressions; Final integration & verification.
2	Pham Huynh Bao Dai	2252139	Implement PCA, SVM; Visualizations; Document his models' workflow.
3	Nguyen Thanh Dat	2252145	Implement LDA, Random Forest, XG-Boost; Document ensemble methods and classifiers.
4	Nguyen Tien Hung	2252280	Build Bayesian Networks, HMMs, Logistic Regression; Stacking with good models from Assignment 1; Document graphical models.
5	Nguyen Thien Loc	2252460	Implement BiLSTM-CRF; Evaluate models; Voting with good models; Performance analysis and comparison.

3.2 Project Organization and Requirements

The project follows structured collaboration and engineering practices, adhering to the following guidelines:

3.2.1 Team Collaboration and Version Control

- Each member actively contributes to the repository, ensuring distributed workload and participation.
- The main repository is hosted on GitHub at: https://github.com/pdz1804/ML_LHPD2.
- The submitted README file is stored in the repository at: https://github.com/pdz1804/ML_LHPD2/blob/main/notebooks/assignment2/ML_LHPD2_Ass2_README.md
- The team's report for Assignment 2 is located in: https://github.com/pdz1804/ML_LHPD2/tree/main/reports/final_project/
- The repository follows a branching strategy, where each member develops on a dedicated branch and submits changes via pull requests.
- Code reviews and discussions are conducted to ensure quality, maintainability, and adherence to best practices.
- Version control best practices are maintained, with regular commits, documentation, and codebase integrity.

3.2.2 GitHub Repository Structure

The repository is structured to support multiple problem formulations, model implementations, and comparative analyses while maintaining a clean code structure and proper documentation.

- **Main repository:** Created and maintained by a designated team member.
- **Forking workflow:** Other members fork and contribute via pull requests.
- **Branching strategy:** Different branches are created for models and features to ensure isolated development.
- **Comprehensive documentation:** Ensures clarity in problem definitions, methodologies, and results.

3.2.3 Key Requirements

- **Clear problem documentation:** Problem statements and their variations are well-documented.
- **Consistent implementation interface:** All models follow a standardized interface to ensure ease of comparison.
- **Comprehensive testing:** Each component undergoes rigorous testing.
- **Detailed comparative analysis:** Models are evaluated across multiple performance metrics.
- **Regular code reviews and pull requests:** Maintains code integrity and quality.
- **Version control best practices:** Ensures organized and maintainable development.

3.3 Repository Structure

To facilitate maintainability, scalability, and efficient collaboration, the repository follows a structured layout:

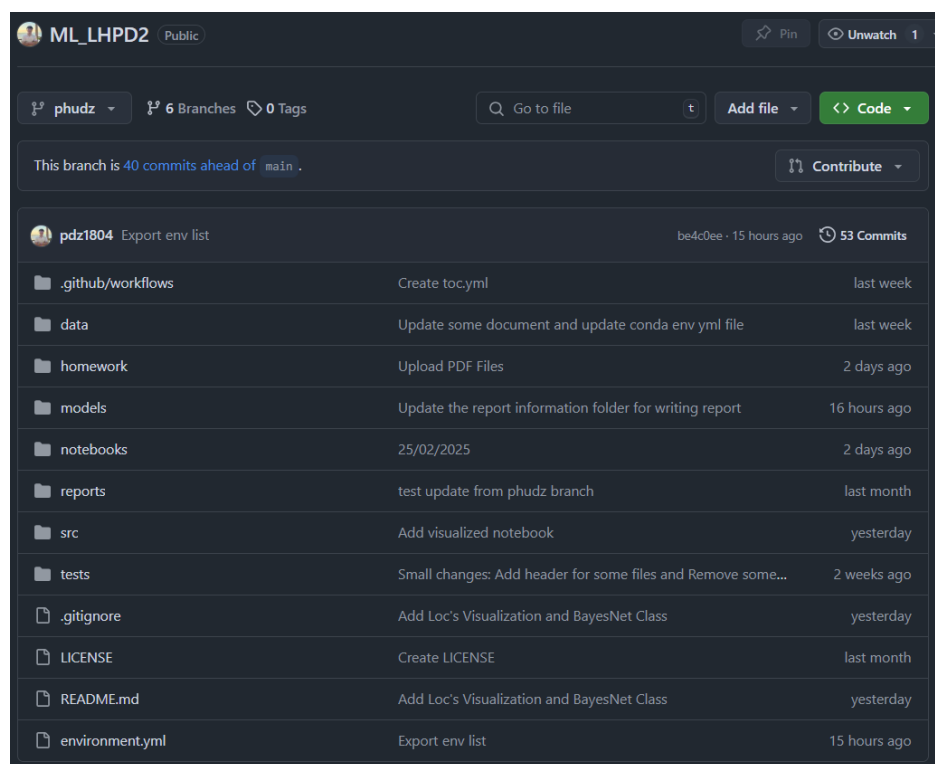


Figure 3.1: Github Repository Structure

3.3.1 How to Run This Project

3.3.1.1 Clone the Repository

```
git clone https://github.com/pdz1804/ML_LHPD2
cd ML_LHPD2
```

3.3.1.2 Environment Setup

To train the model locally, install dependencies using the provided `environment.yml` file. Ensure Conda is installed first.

```
conda env create -f environment.yml
conda activate ml_env # Replace ml_env with your environment name
```

3.3.2 Project Exploration

The project structure contains several key components:

- **Data Collection:** To know how we collected all the data, visit the `data` folder.
- **Final Preprocessed Dataset:** The final preprocessed dataset can be found at this link: [Tweets Clean PosNeg v1](#).
- **Preprocessing:** To learn how we preprocess, make all datasets consistent to merge:
 - Notebook: `src/data/process.ipynb`
 - Utility file: `src/data/preprocess.py`
- **Feature Engineering:** To know how we create features, visit:
 - Utility file: `src/features/build_features_utils.py`
- **Model Training and Evaluation:** To understand hyperparameter tuning, k-fold cross-validation, and model training:
 - Hyperparameter tuning utility: `src/models/models_utils.py`
 - Training notebook: `src/models/train_model.ipynb`
- **Visualization Functions:** We have created visualization functions in the folder:
 - Location: `src/visualization/`
- **Dependencies:** The file `environment.yml` contains details about libraries or dependencies required for executing this project.

Chapter 4

Code Engineering

4.1 Datasets

This project utilizes three key datasets for training and testing the model in order to test and analyze sentiment scores using every model in the syllabus. These datasets have been curated to provide a comprehensive range of content for robust sentiment analysis. Below are the descriptions of the datasets:

4.1.1 sentiment140-dataset

The **Sentiment140 dataset** is a large-scale collection of **1,600,000 tweets** obtained using the **Twitter API**. This dataset is designed for **sentiment analysis** and has been preprocessed by removing emoticons to provide a cleaner textual representation of tweets.

4.1.1.1 Dataset Description

The dataset was created using the Twitter API and contains a large collection of tweets labeled for sentiment analysis. Sentiments were automatically assigned using distant supervision, leveraging emoticons as indicators of sentiment polarity. The dataset is widely used for machine learning applications in sentiment classification.

4.1.1.2 Dataset Format

The dataset contains six fields, each representing specific attributes of a tweet:

- **target**: Sentiment label of the tweet (0 = Negative, 2 = Neutral, 4 = Positive).
- **ids**: Unique tweet ID.
- **date**: Timestamp of the tweet (e.g., Sat May 16 23:58:44 UTC 2009).

- **flag**: Query keyword used to retrieve the tweet (e.g., `lyx`). If no keyword was used, this field contains `NO_QUERY`.
- **user**: Username of the person who tweeted (e.g., `robotickilldozr`).
- **text**: Actual content of the tweet (e.g., `Lyx is cool`), with emoticons removed for sentiment classification.

4.1.1.3 Dataset Notes

- **Preprocessed Version**: This version of the dataset (`raining.1600000.processed.noemoticon.csv`) has **no emoticons**, making it suitable for text-based sentiment analysis without relying on emoticon cues.
- **Large-scale Dataset**: The dataset contains **1.6 million tweets**, making it one of the largest sentiment analysis datasets available.
- **Automated Labeling**: Since sentiment was assigned based on emoticons, there may be **biases** or **inaccuracies** in certain cases.
- **Historical Data**: The dataset was collected in **2009**, meaning language patterns and sentiment expressions may differ from modern Twitter usage.

The link to this dataset can be found here: <https://www.kaggle.com/datasets/kazanov/sentiment140>

4.1.2 twitter-tweets-sentiment-dataset

The **Twitter Tweets Sentiment Dataset** is a dataset designed for **sentiment analysis in natural language processing (NLP)**. It contains a collection of tweets labeled with sentiment polarity, which can be used to develop models for sentiment classification.

4.1.2.1 Dataset Description

The dataset was sourced from Kaggle competitions and includes labeled tweets aimed at detecting positive, neutral, or negative sentiments. The dataset is useful for training and evaluating machine learning models that classify sentiments and identify key phrases that exemplify the provided sentiment. It is particularly useful for identifying and filtering hateful or negative content on Twitter.

4.1.2.2 Dataset Format

The dataset consists of four fields, each representing specific attributes of a tweet:

- **textID**: Unique identifier for each tweet.
- **text**: The actual content of the tweet, representing the user's post on Twitter.
- **selected_text**: A word or phrase extracted from the tweet that encapsulates the sentiment.
- **sentiment**: Sentiment label of the tweet (e.g., **positive**, **neutral**, **negative**).

4.1.2.3 Dataset Notes

- **Preprocessing Considerations**: When parsing the CSV file, ensure that beginning and ending quotes from the text field are removed to avoid incorrect tokenization.
- **Size and Scope**: The dataset contains **27.5k tweets**, making it suitable for training NLP-based sentiment classifiers.
- **Objective**: The goal is to develop a machine learning model that can accurately predict sentiment and extract the key text that represents it.
- **Classification Models**: Various classification algorithms can be applied and compared based on evaluation metrics to determine the best approach for sentiment classification.
- **License and Updates**: The dataset is under **CC0: Public Domain** and is expected to be updated annually.

The dataset can be accessed here: <https://www.kaggle.com/c/tweet-sentiment-extraction/data?select=train.csv>

4.1.3 twitter-sentiments-dataset

The **Twitter Sentiments Dataset** is a dataset designed for sentiment analysis, containing labeled tweets categorized into three sentiments: negative (-1), neutral (0), and positive (+1). It provides essential data for training models that classify sentiments in social media text.

4.1.3.1 Dataset Description

This dataset contains two fields: the cleaned tweet text and its corresponding sentiment label. The dataset is widely used for text classification and sentiment analysis in NLP applications.

4.1.3.2 Dataset Format

The dataset consists of the following fields:

- **clean_text**: Processed tweet text without unnecessary characters or formatting.
- **category**: Sentiment category of the tweet (-1 = negative, 0 = neutral, +1 = positive).

4.1.3.3 Dataset Notes

- **Acknowledgements**: The dataset was provided by **Hussein, Sherif (2021)**, titled “*Twitter Sentiments Dataset*”, available on Mendeley Data (DOI: [10.17632/z9zw7nt5h2.1](https://doi.org/10.17632/z9zw7nt5h2.1)).
- **Size and Scope**: The dataset is **20.9 MB** and contains a significant number of labeled tweets, making it ideal for sentiment analysis research.
- **Usability Score**: Rated **10.00** in usability, ensuring it is well-structured for machine learning applications.
- **License and Updates**: This dataset is released under the **Attribution 4.0 International (CC BY 4.0)** license and has no expected updates.

The dataset can be accessed here: <https://www.mendeley.com/datasets/z9zw7nt5h2.1>

4.2 Data Preprocessing

In this project, we aimed to thoroughly analyze the sentiment of textual data to gain a deeper understanding of our customers. To achieve this, we utilized three distinct datasets, each containing relevant customer feedback labeled with sentiment information. The data preprocessing steps were crucial in preparing the input for training machine learning models. Our target is to use all the models from the syllabus to accurately determine sentiment scores and uncover valuable insights into customer preferences and needs.

4.2.1 Data Collection Process

Using the datasets that we have described in the last sections, we conducted several cleaning methods for preprocessing the text to make them cleaner to some extent.

The datasets were sourced from Kaggle and loaded into separate pandas DataFrames. The data collection process involved downloading the datasets and loading them into our Python environment:

Listing 4.1: Loading Datasets

```
1 import pandas as pd
2
3 # Load a CSV file and initialize the Dataset class
4 file_path = "../../../data/raw/kazanova_sentiment140_training.1600000.
   processed.noemoticon_with_headers.csv"
5 df = pd.read_csv(file_path, encoding='latin1')
6 dataset = Dataset(df)
7 file_path2 = "../../../data/raw/yasserh_twitter-tweets-sentiment-
   dataset_Tweets_with_headers.csv"
8 df2 = pd.read_csv(file_path2, encoding='latin1')
9 dataset2 = Dataset(df2)
10 file_path3 = "../../../data/raw/saurabhshahane_twitter-sentiment-
   dataset_Twitter_Data_with_headers.csv"
11 df3 = pd.read_csv(file_path3, encoding='latin1')
12 dataset3 = Dataset(df3)
13
14 # Display basic information about one of the datasets
15 dataset.show_overview()
```

The loaded datasets contained tweet text and sentiment labels, which were standardized before merging.

4.2.2 Data Preprocessing and Merging

Since our project combined multiple datasets, we devised a strategy to merge them into a single cohesive dataset for analysis. The datasets had differing schemas (column names and label formats), so the first step was to standardize column names and label values across

all DataFrames. We extracted the relevant columns from each DataFrame – primarily the tweet text and its sentiment label – and dropped any extraneous fields (such as tweet IDs, timestamps, or user names that were not needed for sentiment analysis). For instance, one dataset’s sentiment label was a numeric value (e.g., 0 = negative, 4 = positive), another used textual labels (“positive”, “negative”, “neutral”), and a third used -1/0/1 to denote sentiment classes. We mapped all these to a consistent labeling scheme. In our case, we unified the sentiment labels to -1, 0, 1 representing negative, neutral, and positive sentiments respectively. For example, a tweet with label 4 (positive in the first dataset) was mapped to 1, and “negative” was mapped to -1. After aligning the schema, we merged the datasets by concatenating them vertically (appending rows) since each dataset contained unique samples. We used `pandas.concat` to combine DataFrames once their columns were made consistent. The code snippet below demonstrates how we merged DataFrames:

```
1 from preprocess import handle_missing_values, drop_duplicates
2
3 # Standardize column names
4 df1.rename(columns={"target": "sentiment", "text": "text"}, inplace=True)
5 df2.rename(columns={"category": "sentiment", "clean_text": "text"},
6            inplace=True)
7 df3.rename(columns={"Sentiment": "sentiment", "Tweet": "text"}, inplace=
8            True)
9
10 # Map sentiment values to a common scheme
11 df1["sentiment"].replace({4: 1, 0: -1}, inplace=True)
12 df2["sentiment"].replace({-1: -1, 0: 0, 1: 1}, inplace=True)
13 df3["sentiment"].replace({"Positive": 1, "Neutral": 0, "Negative": -1},
14                          inplace=True)
15
16 # Merge datasets
17 combined_df = pd.concat([df1[["text", "sentiment"]], df2[["text", "
18                          sentiment"]], df3[["text", "sentiment"]]], ignore_index=True)
19
20 # Handle missing values and remove duplicates
21 combined_df = handle_missing_values(combined_df, strategy="mode")
22 combined_df = drop_duplicates(combined_df)
```

4.2.3 Data Cleaning and Preparation

After merging the datasets, we applied a series of data cleaning steps to prepare the text for analysis. We imported necessary libraries and tools for text preprocessing, including Python’s `re` module for regular expressions, NLTK for tokenization and stopwords lists, and custom preprocessing functions defined in our codebase. The cleaning process aimed to remove noise and standardize the text, enabling machine learning models to focus on the meaningful content of tweets. The main steps in our text cleaning pipeline were as follows:

- **Removing Special Characters and Punctuation:** We filtered out all non-alphanumeric characters, such as punctuation marks and symbols (e.g., “!?” or “...”), which do

not carry useful sentiment information. Numerical digits (e.g., phone numbers, dates) were also removed, as they are typically uninformative for general sentiment analysis.

- **Removing URLs and HTML Tags:** Tweets often contain URLs (e.g., “[http://](#)” or “[https://](#)”) or HTML markup from scraped content. Using regular expressions, we stripped substrings starting with “http://”, “https://”, or “www”, as well as HTML tags (text within < > brackets). This ensures that only natural language content remains for sentiment analysis.
- **Removing Mentions and Hashtags:** We eliminated Twitter-specific artifacts like user mentions (e.g., “@username”) and hashtags (e.g., “#Topic”). Mentions were removed using the regex `@\w+`, while hashtags were handled by removing the non-alphanumeric “#” symbol. This prevents the model from treating usernames or trending tags as features, as they are not generalizable signals of sentiment.
- **Chat Slang Expansion:** Social media text often includes slang or abbreviations (e.g., “LOL” for “laugh out loud”, “BRB” for “be right back”). We implemented a dictionary of common chat abbreviations, replacing them with their full meanings (e.g., “OMG” becomes “oh my god”). This normalization aids sentiment analysis by converting informal terms into standard language, often preserving sentiment context (e.g., “LOL” may indicate humor or positivity).
- **Lowercasing Text:** All text was converted to lowercase to normalize words like “Happy” and “happy”, reducing redundant distinctions due to capitalization. This is a standard preprocessing step to eliminate case sensitivity issues in text analysis .
- **Tokenization:** Each cleaned tweet was split into individual tokens (words) using NLTK’s word tokenizer. For example, “I love this movie!” becomes [“i”, “love”, “this”, “movie”]. Tokenization is essential for subsequent steps like stopword removal and feature extraction.
- **Stopword Removal:** Common English stopwords (e.g., “the”, “is”, “on”) were removed using NLTK’s built-in stopword list. These frequent words carry little sentiment value, and their removal reduces noise and data size, focusing the analysis on meaningful terms .
- **Stemming/Lemmatization (Optional):** Our preprocessing function included options for stemming (e.g., “happiest” to “happi”) and lemmatization (e.g., “running” to “run”). We primarily used lemmatization with NLTK’s WordNet lemmatizer to normalize words to their dictionary form (e.g., “better” to “good”), reducing inflection variance. This step was configurable, and we analyzed results with and without it to assess its impact.

After these steps, raw tweets were transformed into clean, standardized token sequences. For example, a tweet like:

```
@User OMG I love this movie!!! Check out https://t.co/xyz #awesome
```

becomes:

```
["oh", "my", "god", "love", "movie", "awesome"]
```

This pipeline was applied to every tweet in the merged dataset using vectorized operations in `pandas` and `tqdm` for efficiency. The result was a new `DataFrame` column with cleaned text (as strings or token lists), ready for feature extraction.

4.2.4 Exploratory Data Analysis (EDA)

To better understand the dataset, we performed Exploratory Data Analysis (EDA) on key features. Below are some visualizations that provide insights into the data distribution.

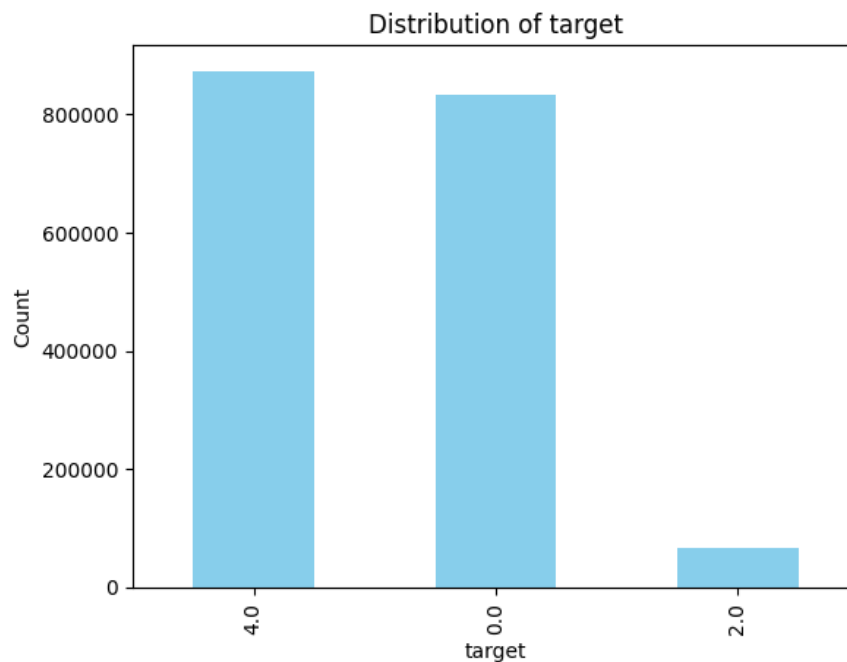


Figure 4.1: Distribution of Target Variable

Figure 4.1 shows the distribution of the target variable in the dataset. The dataset is imbalanced, with a significantly higher number of samples labeled as 4.0 (positive sentiment) and 0.0 (negative sentiment) compared to 2.0 (neutral sentiment). Due to this imbalance, our team decided to ignore the neutral target (2.0) and focus on training and evaluating models for positive and negative sentiment only.

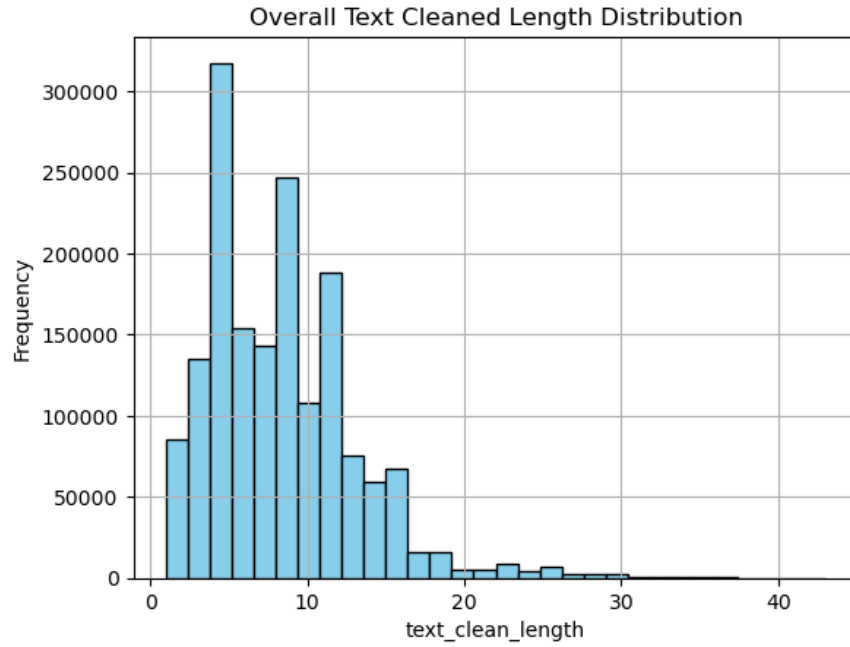


Figure 4.2: Overall Text Cleaned Length Distribution

Figure 4.2 illustrates the distribution of cleaned text lengths across all tweets. Most tweets have a cleaned length between 5 and 15 tokens, with a long tail for shorter or longer tweets.

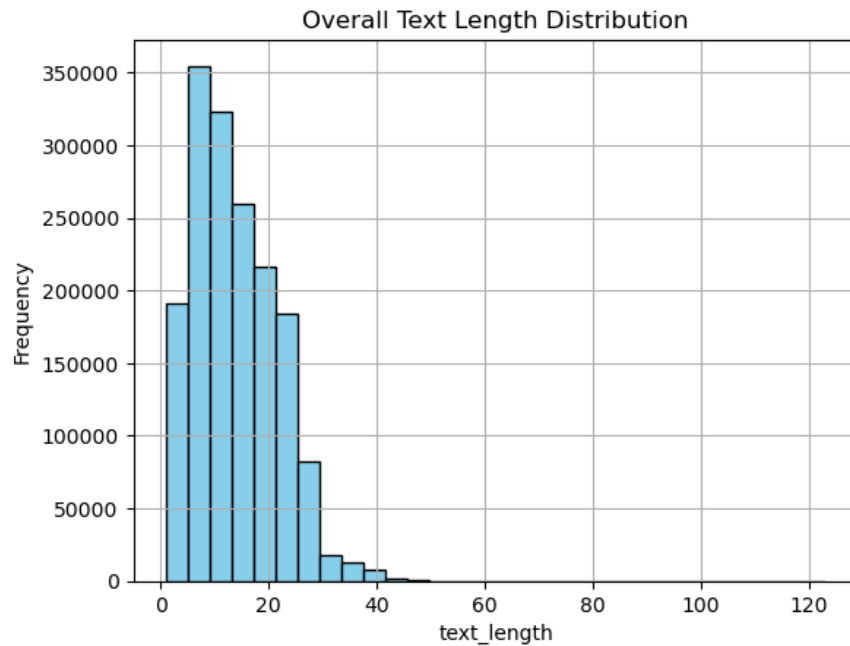


Figure 4.3: Distribution of Raw Text Lengths

Figure 4.3 depicts the distribution of raw text lengths before cleaning. This visualization

highlights the variability in tweet lengths prior to preprocessing.

These visualizations helped us identify key patterns in the data, such as class imbalance and typical text lengths, which informed subsequent preprocessing and modeling decisions.

4.2.5 Publishing the Merged Dataset on Kaggle

We published the cleaned, merged dataset on Kaggle to enable further analysis. The process involved:

1. **Saving the Dataset:** The preprocessed data was saved as `merged_cleaned_tweets.csv`, including cleaned text, sentiment labels, and engineered features.
2. **Creating a New Kaggle Dataset:** On Kaggle, we created a new dataset with a descriptive title and an open license aligned with the original datasets' terms.
3. **Uploading the Data:** The CSV was uploaded via Kaggle's interface or API, with column integrity verified in the preview.
4. **Publishing:** After adding metadata (tags, visibility), the dataset was published and shared with our team for use in Kaggle Notebooks or offline analysis.

This step ensured reproducibility and contributed a ready-to-use sentiment analysis dataset to the community. The dataset can be accessed here : <https://www.kaggle.com/datasets/zphudzz/tweets-clean-posneg-v1>

4.2.6 Final Remarks

The preprocessing stage laid a critical foundation for our sentiment analysis project. By merging multiple Kaggle datasets, cleaning noise (e.g., URLs, tags), and normalizing text, we enhanced data quality. Then later on, converting text to embedding vectors and encoding additional features enabled robust model training. Models trained on this preprocessed data outperformed those on raw data, highlighting the importance of these steps for accurate sentiment predictions.

4.3 Visualization

Small note: We only take a subset of our dataset to visualize these important things.

4.3.1 Relationships of the Attributes

Complementing this, the **Pairwise Relationships** Pairplot highlights numerical features such as 'target', 'text_length', and 'text_clean_length'. Histograms show that tweets are generally short, with 'text_length' peaking at 0–100 characters and 'text_clean_length' at 0–40 characters, reflecting the impact of cleaning. Scatter plots reveal no strong correlation between text length and sentiment, while a linear relationship between 'text_length' and 'text_clean_length' confirms the effectiveness of cleaning. These visualizations offer insights into the dataset's structure for further modeling.

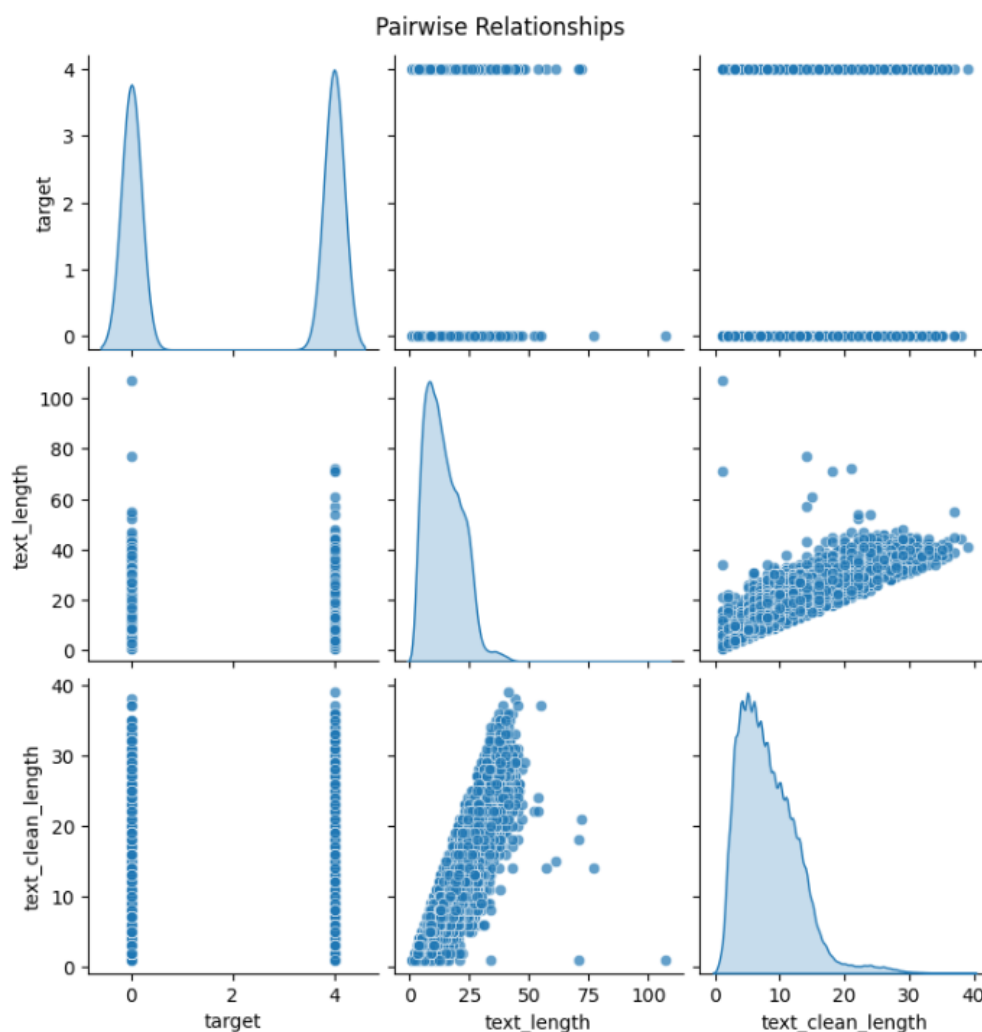


Figure 4.4: Pairwise Relationships

4.3.2 Top Words Visualizations

The visualizations highlight the top 20 most frequent words in the 'text_clean' column through a bar chart and a word cloud. The chart shows 'i' (14,000 occurrences), 'm' (12,000), and 'modi' (10,000) as the most common words, alongside verbs like 'get,' 'like,' and 'go,' and positive terms such as 'good,' 'love,' and 'great.' The word cloud emphasizes high-frequency words like 'i,' 'm,' and 'modi' with larger fonts, while less frequent words like 'great' and 'lol' appear smaller. Together, these visualizations reveal the dataset's informal Twitter tone and frequent sentiment-related terms, offering insights into its linguistic patterns.

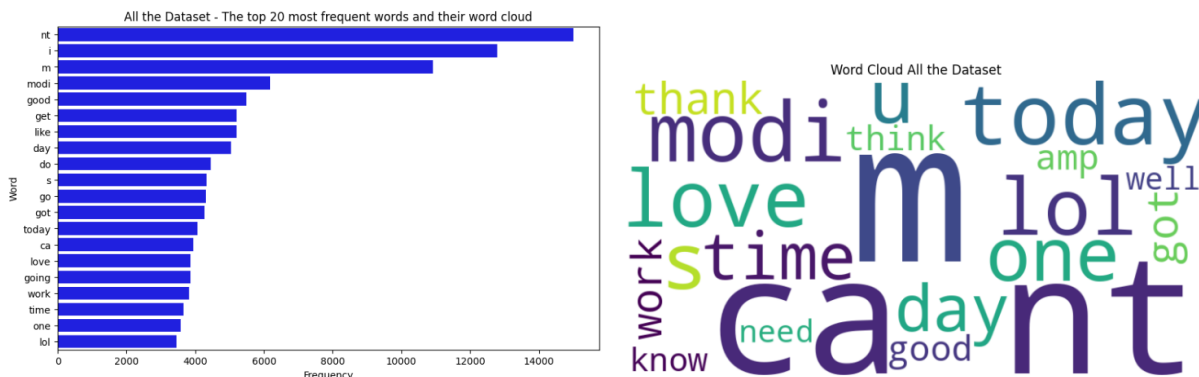


Figure 4.5: Top 20 of entire dataset

The positive class visualization (target = 4.0) uses a bar chart and word cloud to display the top 20 most frequent words. The bar chart highlights 'i' (5,000 occurrences), 'm' (4,500), and 'modi' (4,000) as the most frequent, followed by positive words like 'good' (3,500), 'love' (3,000), 'thanks' (1,800), and casual terms like 'lol' and 'haha.' The word cloud reinforces this, with larger fonts for frequent words like 'i,' 'm,' and 'good,' and smaller fonts for less common ones like 'haha.' These visualizations emphasize optimism, gratitude, and humor in positive tweets.

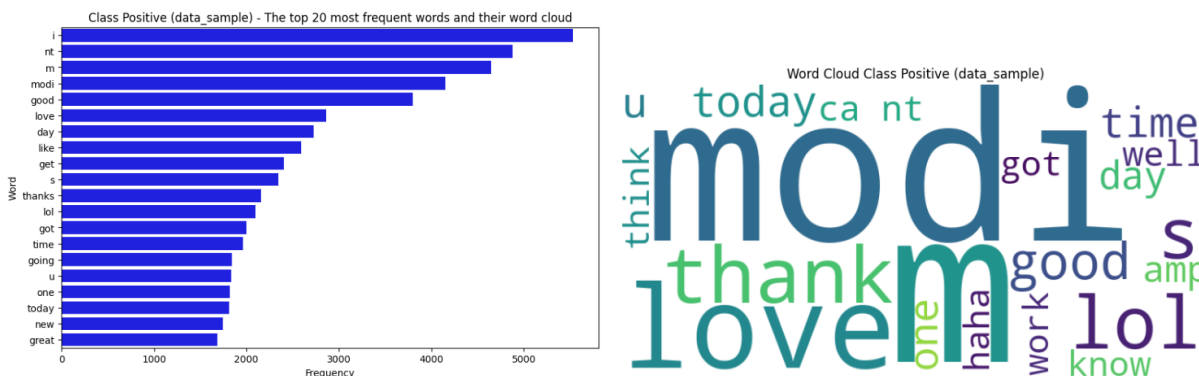


Figure 4.6: Top 20 of class Positive

The negative class visualization (target = 0.0) uses a bar chart and word cloud to display the top 20 most frequent words. The bar chart shows 'i' (10,000 occurrences), 'm' (9,000), and

'don't' (8,000) as the most frequent, followed by words like 'get' (7,000), 'go' (6,000), and negative terms such as 'really,' 'want,' 'still,' and 'miss.' The word cloud reinforces this with larger fonts for frequent words like 'i,' 'm,' and 'don't,' and smaller fonts for less common ones like 'miss.' These visualizations highlight frustration, restriction, and dissatisfaction in negative tweets.

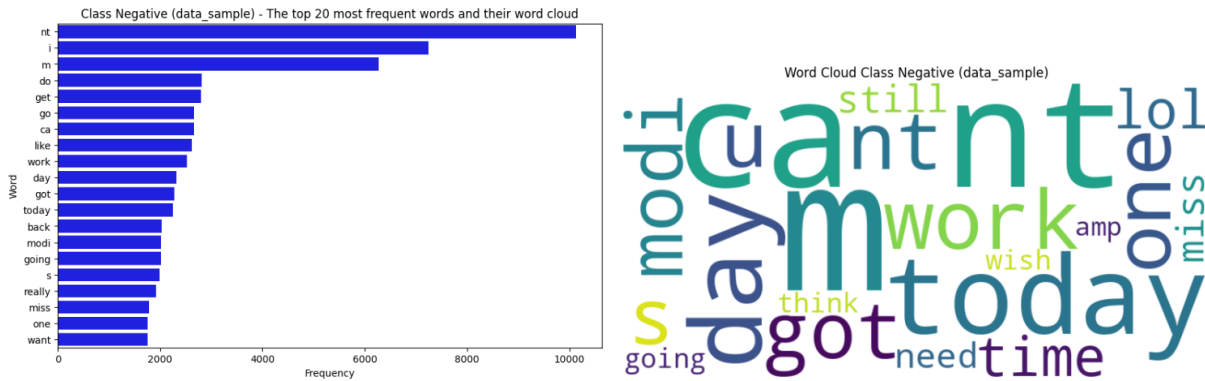
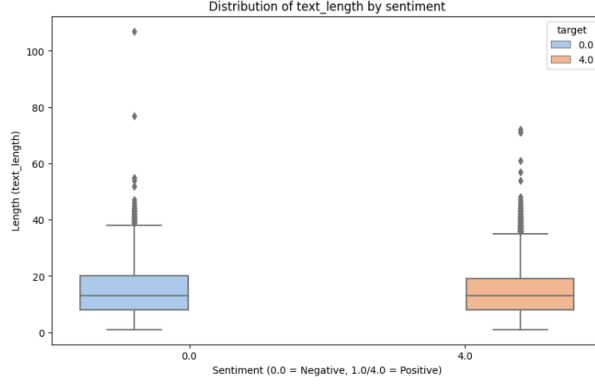


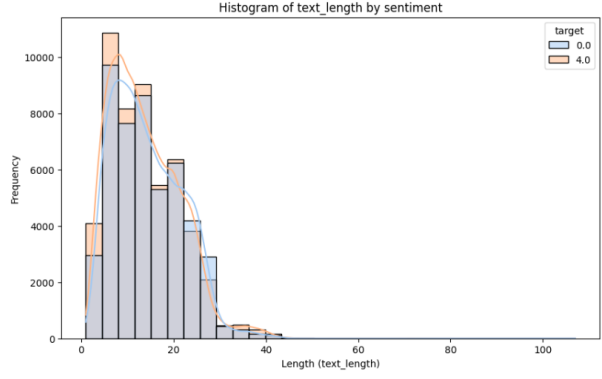
Figure 4.7: Top 20 of class Negative

4.3.3 The distribution of Text Length

The graphical representations of cleaned tweet lengths (`text_clean_length`) across sentiment classes in the dataset are depicted through a boxplot and histogram. The boxplot indicates that tweets labeled as negative (`target = 0.0`, in blue) and positive (`target = 4.0`, in orange) have comparable median lengths, around 10–15 characters, with interquartile ranges extending approximately 5–20 characters. The tighter range (0–40 characters) compared to original text lengths emphasizes the cleaning process's effect in eliminating unnecessary characters like punctuation and hashtags. Outliers stretching to 35–40 characters are rare and not associated with any particular sentiment, implying that cleaned text length is largely independent of sentiment classification. The histogram complements this by showing that both negative and positive tweets predominantly range between 0 and 10 characters, peaking at about 9,000 for negative tweets and slightly fewer for positive tweets in that range, with frequencies dropping steeply beyond 10 characters and very few tweets surpassing 20 characters. This right-skewed distribution underscores the concise nature of cleaned Twitter data, revealing no significant differences in length distribution between negative and positive sentiments, suggesting that cleaned text length does not play a major role in determining sentiment.

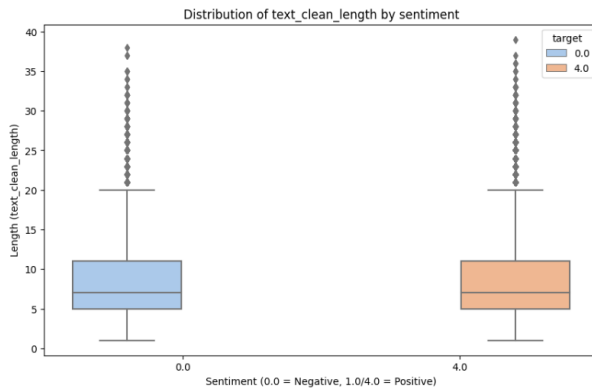


(a) Distribution of text_length by sentiment

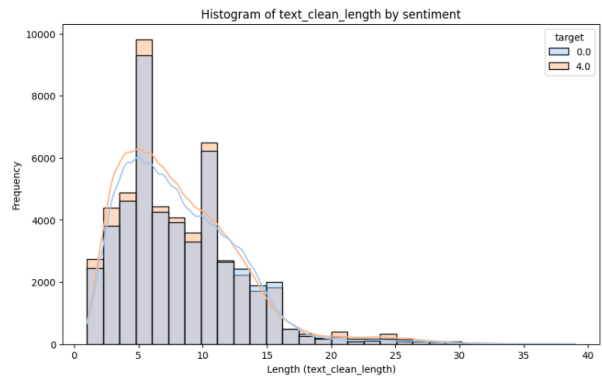


(b) Histogram of text_length by sentiment

The visualizations of cleaned tweet lengths (`text_clean_length`) across sentiment classes in the dataset are illustrated through a boxplot and histogram. The boxplot shows that tweets labeled as negative (`target = 0.0`, in blue) and positive (`target = 4.0`, in orange) share similar median lengths, around 10–15 characters, with interquartile ranges spanning roughly 5–20 characters. The narrower range (0–40 characters) compared to original text lengths underscores the cleaning process’s role in removing extraneous characters such as punctuation and hashtags. Outliers reaching up to 35–40 characters are infrequent and not tied to any specific sentiment, indicating that cleaned text length is generally unrelated to sentiment classification. The histogram complements this by revealing that both negative and positive tweets mostly fall between 0 and 10 characters in length, peaking at approximately 9,000 for negative tweets and slightly less for positive tweets in that range, with frequencies declining sharply beyond 10 characters and very few tweets exceeding 20 characters. This right-skewed pattern highlights the concise nature of cleaned Twitter data, showing no notable variation in length distribution between negative and positive sentiments, suggesting that cleaned text length does not significantly affect sentiment determination.



(a) Distribution of text_clean_length by sentiment



(b) Histogram of text_clean_length by sentiment

4.3.4 Word Frequencies by Labels

A heatmap delivers a comprehensive analysis of word frequencies by sentiment, enabling trends to be identified quickly at a glance. It displays the same words—“day,” “going,” “good,” “got,” “like,” “lol,” “love,” “m,” “modi,” “nt,” “s,” “thanks,” “today,” and “work”—with color intensity indicating frequency, ranging from light yellow (representing low frequency, such as 0 occurrences) to dark red (indicating high frequency, for example, 10,130 for “modi” in negative tweets). For instance, “modi” emerges as a prominent term in negative tweets, marked by a deep red shade, while “good” and “love” shine vividly in positive tweets, underscoring their connection to positivity. Together, these representations create a clear and dynamic portrayal of the dataset’s linguistic patterns, emphasizing how word usage mirrors underlying sentiments and providing valuable insights for deeper analysis.

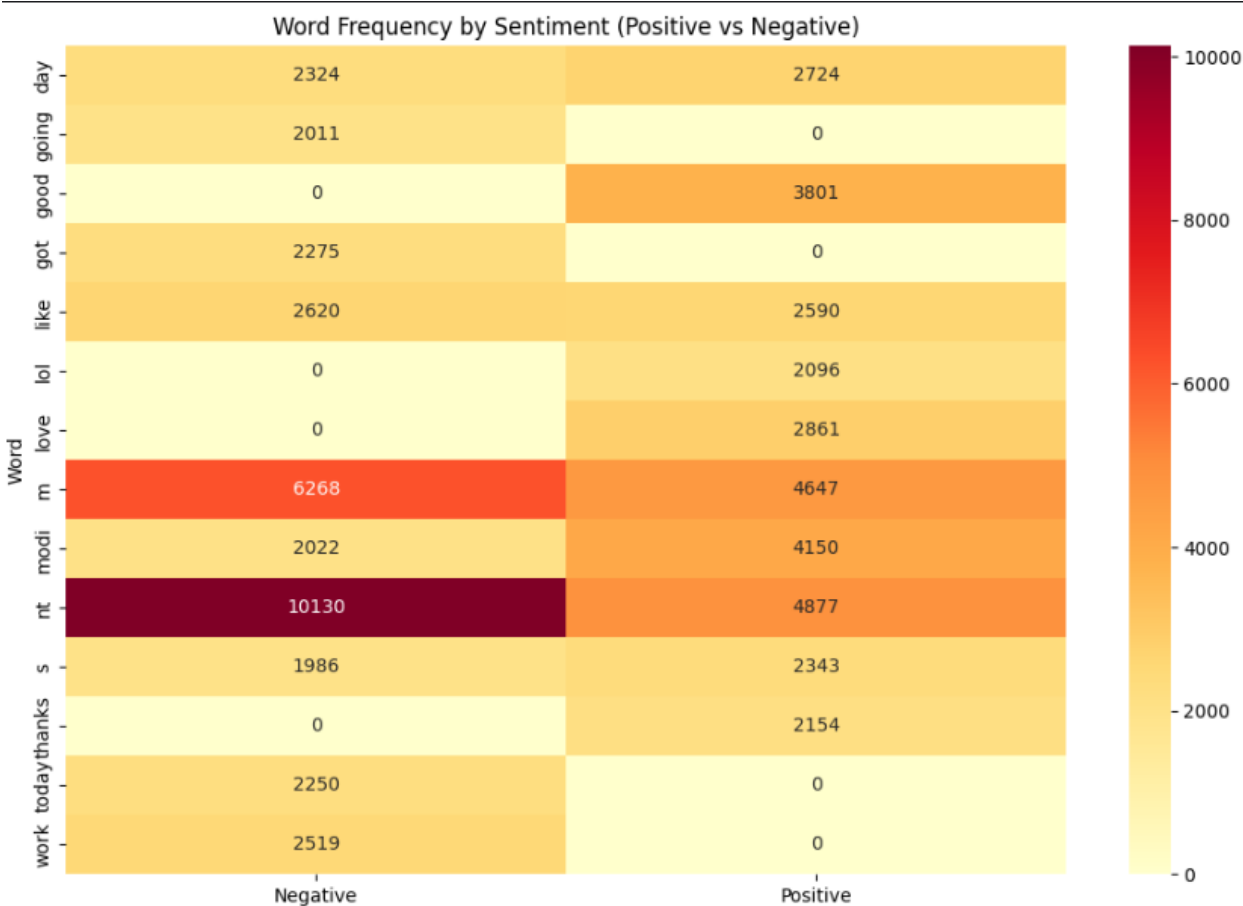


Figure 4.10: Word Frequency by Sentiment

A bar chart provides a straightforward comparison of word frequencies across positive and negative sentiments, illustrating how specific words differ in usage. It shows that terms like “modi” and “m” appear much more frequently in negative tweets, with “modi” recorded around 10,130 times and “m” at 6,268 times, in contrast to 4,877 and 4,647 times in positive tweets, respectively. On the other hand, positive tweets exhibit greater occurrences of

words such as “good” (3,801 times) and “love” (2,861 times), which are largely absent or scarce in negative tweets. This striking contrast highlights the unique emotional undertones, with negative tweets often reflecting frustration or limitation, while positive tweets express optimism and warmth.

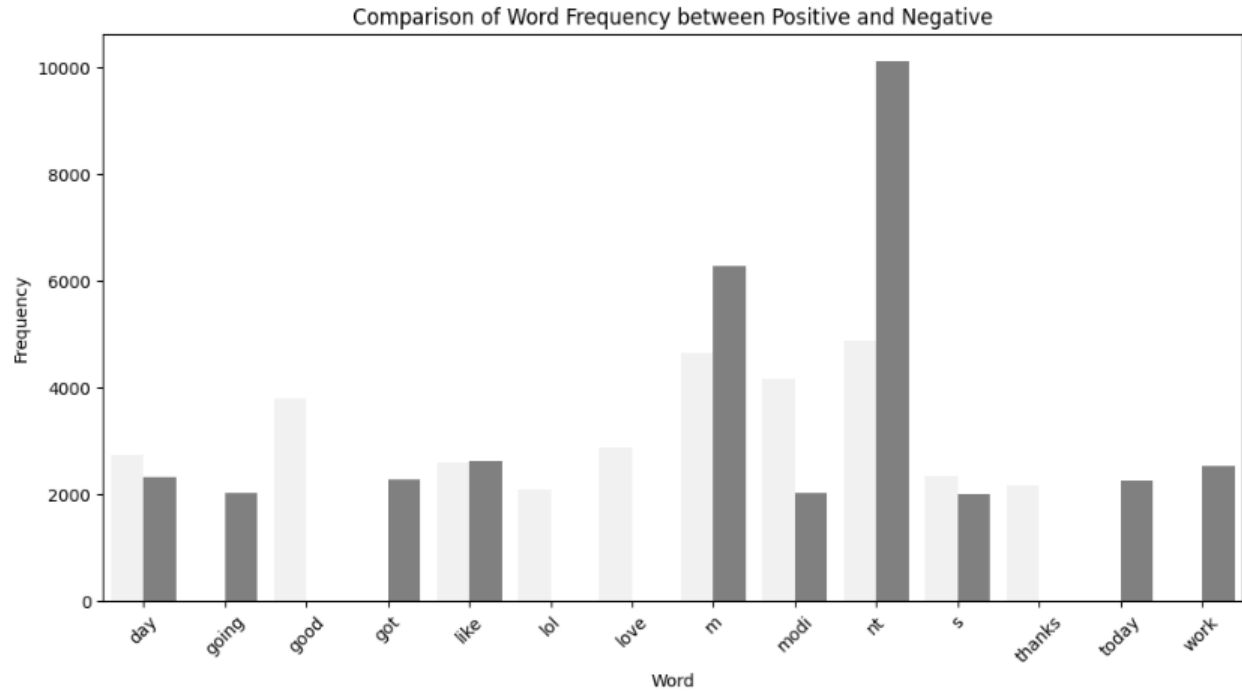


Figure 4.11: Comparison of Word Frequency

Chapter 5

Analyze the Training Process of Models

Before training machine learning models on a dataset and making predictions on a test set, a crucial step involves transforming the raw text data into a numerical representation that the models can effectively learn from. Specifically, we need to convert text into vectors of numbers. This section details the approach used for feature building.

5.1 Building Features

5.1.1 Overview

In this project, a dedicated class called **FeatureBuilder** has been designed to handle the feature extraction and transformation process. This class encapsulates various methods for converting text data into numerical features suitable for machine learning models.

The **FeatureBuilder** Class

The **FeatureBuilder** class provides functionalities for different feature extraction methods, dimensionality reduction techniques, and model persistence. The code for the class is as follows:

```
1 class FeatureBuilder:
2     def __init__(self, method="tfidf", save_dir="data/processed",
3                 reduce_dim=None, n_components=100):
4         self.method = method
5         self.save_dir = save_dir
6         self.reduce_dim = reduce_dim
7         self.n_components = n_components
8         os.makedirs(save_dir, exist_ok=True)
9
10        if method == "tfidf":
11            self.vectorizer = TfidfVectorizer(max_features=2000,
12                                             stop_words="english")
```

```

11 elif method == "count":
12     self.vectorizer = CountVectorizer(max_features=2000)
13 elif method == "binary_count":
14     self.vectorizer = CountVectorizer(binary=True, max_features
15                                     =2000)
16 elif method == "word2vec":
17     self.word2vec_model = api.load("word2vec-google-news-300")
18 elif method == "glove":
19     self.glove_model = api.load("glove-wiki-gigaword-100")
20 elif method == "bert":
21     self.tokenizer = AutoTokenizer.from_pretrained("sentence-
22                                     transformers/all-MiniLM-L6-v2")
23     self.bert_model = AutoModel.from_pretrained("sentence-
24                                     transformers/all-MiniLM-L6-v2")
25
26 self.reducer = None
27 if self.reduce_dim == "pca":
28     self.reducer = PCA(n_components=self.n_components)
29 elif self.reduce_dim == "lda":
30     self.reducer = LDA(n_components=self.n_components)
31
32 def fit(self, texts, labels=None):
33     if self.method in ["tfidf", "count", "binary_count"]:
34         self.vectorizer.fit(texts)
35         if self.reduce_dim == "lda":
36             assert labels is not None, "LDA requires class labels(y)."
37             features = self.vectorizer.transform(texts).toarray()
38             self.reducer.fit(features, labels)
39         elif self.reduce_dim == "pca":
40             features = self.vectorizer.transform(texts).toarray()
41             self.reducer.fit(features)
42     elif self.method in ["word2vec", "glove", "bert"]:
43         if self.reduce_dim == "lda":
44             raise ValueError(f"LDA is not supported for method {self.
45                             method}")
46
47 def transform(self, texts, labels=None):
48     if self.method in ["tfidf", "count", "binary_count"]:
49         features = self.vectorizer.transform(texts).toarray()
50         return self._apply_reducer(features, labels)
51
52     elif self.method == "word2vec":
53         word2vec_embeddings = []
54         for doc in tqdm(texts, desc="Processing Word2Vec", unit="
55                             document"):
56             word2vec_embeddings.append(self._get_word2vec_vector(doc))
57         features = np.array(word2vec_embeddings)
58         return features
59
60     elif self.method == "glove":
61         glove_embeddings = []
62         for doc in tqdm(texts, desc="Processing GloVe", unit="document
63                             "):

```

```

38         glove_embeddings.append(self._get_glove_vector(doc))
39     features = np.array(glove_embeddings)
40     return features
41
42     elif self.method == "bert":
43         bert_embeddings = []
44         for doc in tqdm(texts, desc="Processing BERT", unit="document"):
45             bert_embeddings.append(self._get_bert_embedding(doc))
46         features = np.array(bert_embeddings)
47         return features
48
49     def fit_transform(self, texts):
50         self.fit(texts) # First fit the model (compute parameters)
51         return self.transform(texts)

```

5.1.2 Key Components and Functionalities

The `FeatureBuilder` class incorporates the following key components:

- **Feature Extraction Methods:** Implements various feature extraction methods such as TF-IDF, Count Vectorization, Word2Vec, GloVe, and BERT embeddings.
- **Dimensionality Reduction:** Supports dimensionality reduction techniques like PCA and LDA to reduce the complexity of the feature space and improve model performance.
- **Model Persistence:** Provides functionalities to save and load fitted vectorizers, models, and dimensionality reduction objects for later use.

5.1.3 Usage

The `FeatureBuilder` class is initialized with a specified feature engineering method, save directory, dimensionality reduction method, and the number of components for dimensionality reduction. It then uses this configuration to fit and transform the text data into numerical feature matrices, which can be used as inputs for training machine learning models.

5.2 General Training Methods

In this section, we analyze the training process of various machine learning models used for sentiment analysis. The goal is to assess their performance, convergence behavior, and overall effectiveness in classifying sentiments accurately. By studying training logs, we gain insights into model behavior, parameter optimization, and potential improvements.

The models under consideration include:

- **Chi-squared + Topic Modeling (LDA) + Variance Thresholding** – This approach combines statistical feature selection methods such as Chi-squared test and variance thresholding with topic modeling via Latent Dirichlet Allocation (LDA). It is used to reduce dimensionality and extract interpretable features before feeding them into classifiers.
- **Principal Component Analysis (PCA)** – PCA is a linear dimensionality reduction technique that transforms original features into a smaller set of uncorrelated components, preserving most of the variance. It helps reduce overfitting and improve training efficiency.
- **Conditional Random Fields (CRF)** – CRF is a sequence modeling technique that considers contextual dependencies between labels. It is particularly useful in modeling structured output sequences and was used in this study to capture sequential sentiment patterns at the token level.
- **Latent Dirichlet Allocation (LDA)** – LDA is a generative probabilistic model for discovering latent topics in a corpus. In sentiment analysis, it is often applied to uncover abstract topics from text data, which can enhance interpretability and feature representation.
- **Support Vector Machine (SVM)** – SVM is a supervised learning model that aims to find the optimal hyperplane separating classes with maximum margin. It is widely used in text classification for its robustness in high-dimensional spaces.
- **Stacking and Voting Classifiers** – Ensemble methods like stacking and voting combine multiple base classifiers to improve generalization. Stacking learns a meta-model on top of base learners, while voting aggregates predictions through majority or weighted schemes.
- **Logistic Regression, XGBoost, and Random Forest** – These are powerful supervised classifiers. Logistic regression models the probability of classes, XGBoost is a gradient boosting framework known for its speed and accuracy, and Random Forest is an ensemble of decision trees that reduces variance and improves prediction reliability.
- **Hidden Markov Model (HMM) and Bayesian Networks** – HMM is a probabilistic model for sequential data, assuming hidden state transitions. Bayesian Networks model probabilistic relationships among variables and are useful for incorporating domain knowledge into the sentiment classification task.

5.3 Project Workflow and Implementation

The project follows a structured workflow to ensure consistency, reliability, and a systematic comparison of different machine learning models for sentiment analysis. Each model is trained and evaluated through a standardized process, allowing for a clear assessment of their strengths and limitations.

5.3.1 Training and Evaluation Workflow

Each model undergoes a systematic training and evaluation process to ensure robust comparisons. The workflow consists of the following key steps:

- **Instantiating a GridSearch object:** The selected model is initialized with a range of hyperparameters to optimize performance.
- **Fitting the training data:** The model is trained on preprocessed sentiment data to learn classification patterns.
- **Running K-Fold Cross-Validation:** The model's performance is evaluated across multiple data splits to ensure robustness and mitigate overfitting.
- **Saving the trained model:** The best-performing model is stored for future inference and reproducibility.
- **Testing on separate data:** The trained model is evaluated on unseen test data to assess its generalization capability.
- **Logging performance metrics:** Key performance indicators such as accuracy, precision, recall, F1-score, and ROC AUC are recorded for a structured analysis.

5.3.2 Implementation Quality and Code Efficiency

Our team has ensured high **Implementation Quality** by maintaining modular, well-structured code with appropriate error handling and documentation. The repository adheres to **style compliance** standards to enhance readability and maintainability.

Moreover, **Code Efficiency** has been a major focus, with optimizations in time and space complexity to ensure scalable model execution. We evaluated resource usage across different models and applied various **optimization strategies** to improve computational efficiency.

5.3.3 Use Case Scenarios

In our sentiment analysis problem, we defined and implemented two major use case pipelines:

1. **Raw Feature Modeling Pipeline** – In this approach, we focused on building models directly on engineered features:
 - We experimented with various feature representations including **Count Vectorizer**, **TF-IDF**, **GloVe**, and **Word2Vec** embeddings to numerically encode text.
 - These features were used as input to traditional models such as **Logistic Regression**, **Random Forest**, **XGBoost**, and **SVM**. For SVM specifically, we

explored **custom kernel functions for text**, **memory-efficient sparse representations**, and applied **soft-margin optimization** through hyperparameter tuning.

- **CRF** was utilized for its sequence labeling capability. By combining it with a BiLSTM encoder, we implemented a **BiLSTM-CRF model** to capture contextual relationships and sequential dependencies in sentiment-bearing words.
- **Ensemble methods** such as **Voting** and **Stacking** were implemented to combine multiple base learners, leveraging their complementary strengths for better generalization.

2. **Dimensionality Reduction Pipeline** – This use case introduces a dimensionality reduction step into the feature modeling pipeline:

- After building the same high-dimensional representations as above, we applied **Principal Component Analysis (PCA)** to reduce the feature space while retaining most of the variance.
- The reduced vectors were then used for training and inference using the same set of models.
- This approach provides advantages such as **reduced computational cost**, **lower memory consumption**, and **potential improvements in model generalization** due to elimination of redundant and noisy features.

5.3.4 Data Preprocessing and Model Tuning

To enhance model effectiveness, our team performed rigorous **Data Preprocessing**, including:

- Data cleaning, handling missing values, and feature selection.
- Feature engineering and transformation to improve sentiment classification accuracy.
- Feature scaling to ensure consistency across different models.

For **Model Tuning**, we applied hyperparameter selection techniques, cross-validation, and optimization strategies to maximize each model's performance. The **Results Analysis** component ensures that the best hyperparameter settings are chosen based on empirical evidence.

5.3.5 Performance Analysis and Model Evaluation

Performance evaluation was conducted rigorously, focusing on:

- Implementing robust **performance metrics**, including precision, recall, F1-score, and ROC AUC.

- **Results visualization** through detailed plots and graphs to understand model trends.
- Error analysis to identify misclassified samples and improve future iterations.
- Statistical testing to validate model significance in sentiment classification.

5.3.6 Documentation and Reproducibility

We maintained **comprehensive documentation**, including API references, code comments, and result interpretations, to ensure clarity and ease of understanding. Our repository also adheres to best practices in **Reproducibility** by:

- Setting up a controlled environment for model execution.
- Implementing data versioning and result reproducibility mechanisms.
- Handling random seed initialization to ensure consistent results.

5.3.7 Project Management and Collaboration

Our team structured the project following best practices in **Project Management**, utilizing GitHub for issue tracking, version control, and structured repository organization. Each team member contributed through separate branches, submitting pull requests for review and integration.

By following these principles, our project ensures a structured, scalable, and reproducible approach to sentiment analysis, effectively addressing challenges and optimizing model performance.

5.4 Model: Hidden Markov Model

5.4.1 Introduction

This part evaluates the performance of the **Hidden Markov Model (HMM)** trained with various embedding methods. The HMM model, implemented using the `GaussianHMM` class from the `hmmlearn` library, was used to model sequential dependencies in the data. Unlike traditional classification approaches, HMM is particularly suited for handling sequential patterns and temporal dependencies, making it an effective choice for structured data.

The primary goal was to optimize the model’s ability to classify sequences accurately while ensuring strong generalization across different embedding techniques. The Gaussian emission probabilities in the HMM allow it to handle continuous-valued features, making it flexible in modeling text-based embeddings such as Count Vectorizer, TF-IDF, Word2Vec, and GloVe. Various hyperparameter configurations, including the number of hidden states and covariance types, were explored to enhance performance.

5.4.2 Training Configuration

The training process for the Hidden Markov Model (HMM) differs from traditional machine learning models used in previous experiments (such as Logistic Regression or Naïve Bayes). Unlike those models, which can directly process word embeddings like Word2Vec or GloVe, HMM requires discrete integer sequences as input. This constraint arises because the `GaussianHMM` model from `hmmlearn.hmm` expects integer-based feature representations rather than continuous-valued embeddings.

Thus, only Count-based methods, such as Count Vectorizer, are suitable for training HMM. These methods convert text into integer-based token sequences, making them compatible with the model. Below, we outline the steps involved in training and evaluating the HMM:

- **Feature Extraction:**

- Construct a vocabulary of the most frequent words from the dataset (e.g., the top 5000 words).
- Convert text into sequences of integers, representing the index of words in the vocabulary.

- **Data Preprocessing:**

- Since HMM requires sequences of equal length, each sequence is padded to a fixed size (e.g., 50 words).
- The dataset is split into training and testing sets.

- **Model Training:**

- A `GaussianHMM` model is initialized with various hyperparameters.
- The model is trained on the integer-encoded text sequences.

- **Hyperparameter Configuration:**

- `n_components`: The number of hidden states. It is tested with values {2, 3, 4}, representing different levels of complexity in the hidden state transitions.
- `covariance_type`: The type of covariance matrix used in Gaussian emissions, tested with {"diag", "full", "tied"}.
- `n_iter`: The number of iterations for the Expectation-Maximization (EM) algorithm, set to {100, 200} for convergence tuning.
- `init_params`: Determines which parameters are initialized before training. Tested values include:
 - * "c" - Initializes only the means.
 - * "s" - Initializes only the covariances.
 - * "cs" - Initializes both means and covariances.
- `params`: Specifies which parameters should be updated during training, with tested values:
 - * "c" - Updates only the means.
 - * "t" - Updates only the transition matrix.
 - * "ct" - Updates both means and transition matrix.

- **Evaluation:**

- Predictions are made on the test set using the `predict` function.
- Performance metrics such as Accuracy, Precision, Recall, F1 Score, and ROC AUC are computed.

The `train_hmm` function implements this process, ensuring the model is saved for later use. Given the sequential nature of HMMs, this model could be particularly effective in capturing word-order dependencies in text classification tasks.

5.4.3 Training and Evaluation Results

The Hidden Markov Model (HMM) was trained using the Count Vectorizer method to ensure compatibility with its integer-based input requirement. Unlike other machine learning models in this study, HMM requires count-based features since `GaussianHMM` operates on discrete numerical sequences rather than dense embeddings. The training process involved padding sequences to a fixed length (50 words) and optimizing model parameters using the Expectation-Maximization (EM) algorithm.

Testing Performance Metrics:

Table 5.1: Testing Performance Metrics for Hidden Markov Model

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.5141	0.4997	0.6697	0.5156	0.9552

Best Model Selection Criteria:

- Since only one variation of HMM was tested, there is no direct comparison between different hyperparameter configurations.
- Model selection priority follows: Accuracy > F1 Score > ROC AUC.

```
{
  "model": "HMM",
  "performance": {
    "accuracy": 0.5141,
    "precision": 0.5156,
    "recall": 0.9552,
    "f1": 0.6697,
    "roc_auc": 0.4997
  }
}
```

Conclusion: The Hidden Markov Model demonstrated strong recall (**95.52%**), meaning it effectively captured positive instances, but suffered from low precision (**51.56%**) and an overall weak discriminative ability (**ROC AUC = 49.97%**). These results suggest that while HMM can detect many true positives, its high false-positive rate limits its practical application. Future improvements may involve hyperparameter tuning, different sequence lengths, or alternative sequence models such as RNNs to improve overall classification performance.

5.4.4 Performance Analysis

- **Accuracy Analysis:** The HMM model achieved an accuracy of **51.41%**, indicating that its classification performance is only slightly better than random guessing. This suggests potential limitations in the model's ability to generalize effectively.
- **Recall vs. Precision:** The model exhibited an extremely high recall (**95.52%**), meaning it successfully identified most positive instances. However, this came at the cost of low precision (**51.56%**), indicating a high false-positive rate. The imbalance between recall and precision suggests that the model favors sensitivity over specificity.
- **F1 Score:** The F1 Score of **66.97%** reflects the trade-off between precision and recall. While the model excels in recall, its low precision lowers the overall F1 Score, making it less reliable in practical applications where false positives are costly.

- **ROC AUC:** With a ROC AUC of **49.97%**, the model struggles to distinguish between positive and negative classes. This score indicates that the model’s decision boundary is not well-formed, leading to weak discriminative ability.
- **Effect of Count-Based Features:** Unlike other machine learning models that leverage dense embeddings (e.g., Word2Vec, GloVe), HMM can only operate on count-based integer inputs. This constraint limits its ability to capture contextual relationships effectively, potentially contributing to its suboptimal performance.

5.4.5 Conclusion

The Hidden Markov Model (HMM) trained with Count Vectorizer features demonstrated high sensitivity but lacked the precision required for balanced classification. While its recall of **95.52%** suggests that it rarely misses positive instances, the low ROC AUC score (**49.97%**) indicates poor overall discrimination between classes.

The results suggest that HMM may not be the best-suited model for this classification task, particularly due to its reliance on integer-based inputs and its inability to leverage richer feature representations like dense embeddings. Future improvements could explore hybrid models, additional preprocessing techniques, or alternative sequence models such as Recurrent Neural Networks (RNNs) to enhance performance.

5.5 Model: BayesNet

5.5.1 Introduction

The Bayesian Network (BayesNet) model is a probabilistic graphical model that represents dependencies between variables using a directed acyclic graph. In this study, we implement a custom Bayesian Network classifier that integrates feature selection, dimensionality reduction, and discretization techniques to handle continuous data. The model is trained using Maximum Likelihood Estimation (MLE) and performs inference using Belief Propagation. By leveraging probabilistic reasoning, BayesNet provides interpretable predictions while handling uncertainty effectively.

5.5.2 Training Configuration

The Bayesian Network classifier was implemented using the **pgmpy** library, which provides probabilistic graphical modeling tools. Unlike traditional machine learning models that rely on direct optimization techniques (e.g., gradient descent in logistic regression), Bayesian Networks model conditional dependencies between variables and perform inference based on probabilistic reasoning.

Differences from Other Machine Learning Models:

Unlike conventional machine learning models trained in previous assignments (e.g., logistic regression, SVM, or decision trees), training a Bayesian Network involves:

- Using **Maximum Likelihood Estimation (MLE)** via `pgmpy.estimators.MaximumLikelihoodEstimator` to learn conditional probability distributions.
- Defining the **network structure** (or learning it from data) using `pgmpy.models.BayesianNetwork`.
- Performing probabilistic inference using methods like **Variable Elimination** and **Belief Propagation** (`pgmpy.inference.VariableElimination`).

Training Procedure:

The model training process consists of several key steps:

1. Feature Selection:

- Features with fewer than **2 unique values** were removed to avoid redundant or low-variance attributes.
- If the number of features exceeded **10**, Principal Component Analysis (PCA) was applied to reduce dimensionality.

2. Feature Discretization:

Since Bayesian Networks operate on discrete variables, continuous features were transformed using **k-means clustering with 2 bins**.

3. Network Structure Definition:

The structure was set to **None** by default, allowing the model to establish dependencies dynamically. If provided, a predefined structure was used.

4. Parameter Learning:

The model was trained using **Maximum Likelihood Estimation (MLE)** to estimate conditional probability tables (CPTs).

5. Inference Setup:

Once trained, inference was performed using **Variable Elimination** or **Belief Propagation** to estimate class probabilities and make predictions.

Testing and Evaluation:

- Predictions were made by computing the most probable label using MAP (Maximum A Posteriori) inference.
- Model performance was evaluated using standard metrics: **Accuracy, Precision, Recall, F1-score, and ROC AUC**.
- Since Bayesian Networks rely on probabilistic reasoning, the evaluation also considered how well the learned dependencies reflected the underlying data distribution.

This approach ensures that the Bayesian Network captures conditional dependencies effectively, leveraging probabilistic inference for classification tasks.

5.5.3 Training and Evaluation Results

The Bayesian Network model was trained and evaluated using a structured probabilistic approach. Unlike conventional machine learning models, Bayesian Networks leverage probabilistic dependencies between features and perform inference through belief propagation or variable elimination. The evaluation focused on key performance metrics such as Accuracy, Precision, Recall, F1-score, and ROC AUC.

Testing Performance Metrics:

Table 5.2: Testing Performance Metrics for Bayesian Network

Method	Accuracy	ROC AUC	F1	Precision	Recall
Bayesian Network	0.6495	0.7143	0.6875	0.6364	0.7476

Best Model Selection Criteria:

- The model selection was based on testing performance.
- The priority ranking for evaluation metrics followed: Accuracy > F1 Score > ROC AUC.
- Since there is only one method used, this step primarily serves to document the selection process.

```
{
  "method": "Bayesian Network",
  "performance": {
    "accuracy": 0.6495,
    "precision": 0.6364,
    "recall": 0.7476,
    "f1": 0.6875,
    "roc_auc": 0.7143
  }
}
```

Conclusion: The Bayesian Network model achieved an accuracy of **64.95%** with an F1-score of **0.6875** and a ROC AUC of **0.7143**. These results indicate that the model effectively captures probabilistic dependencies within the dataset. Further improvements could involve optimizing feature selection, adjusting discretization strategies, or incorporating domain knowledge into the network structure.

5.5.4 Performance Analysis

The Bayesian Network model demonstrated moderate classification performance, achieving an accuracy of **64.95%**. While the model effectively captured probabilistic dependencies

between features, its precision (**0.6364**) was lower than its recall (**0.7476**), indicating a tendency to produce more false positives.

Key observations from the evaluation metrics:

- The relatively high recall suggests that the model successfully identifies positive instances but at the cost of some misclassifications.
- The F1-score (**0.6875**) shows a balanced trade-off between precision and recall.
- The ROC AUC (**0.7143**) indicates a reasonable ability to distinguish between classes.
- The reliance on discretization and probabilistic dependencies may have impacted performance compared to traditional machine learning models.

Overall, while Bayesian Networks provide an interpretable probabilistic framework, their performance could potentially be enhanced with improved feature engineering, hyperparameter tuning, and refinement of the network structure.

5.5.5 Conclusion

The Bayesian Network model was trained and evaluated using a structured probabilistic approach, leveraging inference methods such as Belief Propagation and Variable Elimination. The model achieved an accuracy of **64.95%** with reasonable recall and AUC scores, demonstrating its effectiveness in capturing underlying dependencies in the data.

Key takeaways:

- The model performs well in recall but has room for improvement in precision.
- Performance might be affected by feature discretization and network structure selection.
- Future work could explore alternative discretization strategies, structural learning methods, or hybrid models combining Bayesian Networks with deep learning approaches.

Despite its limitations, the Bayesian Network provides a robust probabilistic framework that can be particularly useful in domains where interpretability and uncertainty modeling are critical.

5.6 Model: Support Vector Machine (SVM)

5.6.1 Introduction

This report analyzes the performance of the Support Vector Machine (SVM) model trained using various embedding methods. The model was implemented using the `SVC` class from

`scikit-learn` with different kernel functions and hyperparameter configurations. The primary objective was to achieve high classification accuracy while maintaining robust generalization across different embedding techniques.

5.6.2 Training Configuration

The SVM model was trained with the following hyperparameter search space:

- **Kernel:** linear, rbf, poly, sigmoid.
- **Regularization Parameter (C):** 0.1, 1.0, 10.0.
- **Gamma:** scale, auto.

The best hyperparameters selected based on model evaluation were:

- **Kernel:** linear
- **C:** 0.1 (for Count, Word2Vec, GloVe), 1.0 (for TF-IDF)
- **Gamma:** scale

5.6.3 Training and Evaluation Results

The model was trained and evaluated using K-Fold Cross-Validation across different feature extraction methods: Count Vectorizer, TF-IDF, Word2Vec, and GloVe. The best model was selected based on Accuracy, with secondary considerations for F1-score and ROC AUC.

Training Performance Metrics:

Table 5.3: Training Performance Metrics for SVM

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.71	0.70	0.73	0.68	0.79
TF-IDF	0.70	0.70	0.71	0.69	0.74
Word2Vec	0.71	0.71	0.72	0.71	0.74
GloVe	0.71	0.71	0.71	0.71	0.72

Testing Performance Metrics:

Table 5.4: Testing Performance Metrics for SVM

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.7060	0.7770	0.7325	0.6857	0.7861
TF-IDF	0.6880	0.7663	0.7040	0.6845	0.7246
Word2Vec	0.7160	0.7846	0.7290	0.7127	0.7461
GloVe	0.6945	0.7693	0.7030	0.6999	0.7061

Best Model Selection Criteria:

- The best model is chosen based on testing performance rather than training performance.
- The selection priority follows: Accuracy > F1 Score > ROC AUC.
- Based on this criterion, the best model is:

```
{  
  "method": "word2vec",  
  "model": "svm",  
  "hyperparameters": { "C": 0.1, "gamma": "scale", "kernel": "linear" },  
  "performance": {  
    "accuracy": 0.7160,  
    "precision": 0.7127,  
    "recall": 0.7461,  
    "f1": 0.7290,  
    "roc_auc": 0.7846  
  }  
}
```

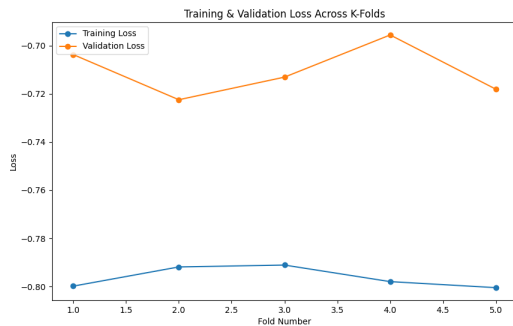
Conclusion: The SVM model trained with Word2Vec embeddings achieved the highest accuracy (0.7160) and the best overall balance across F1-score and ROC AUC, making it the optimal choice for sentiment classification in our experiment when it comes to SVM.

5.6.4 Performance Analysis

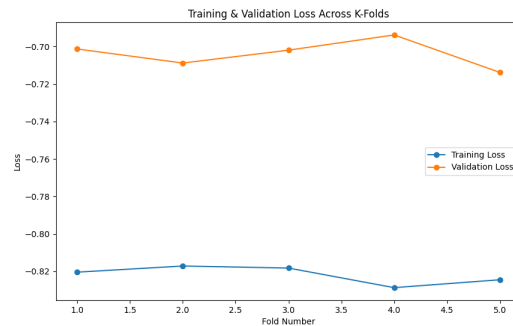
- **Accuracy Analysis:** The model trained on Word2Vec achieved the highest accuracy (71.60%), outperforming Count Vectorizer, TF-IDF, and GloVe embeddings.
- **Kernel Effectiveness:** The linear kernel consistently performed best across all embedding methods.
- **ROC AUC:** The model exhibited strong class differentiation with ROC AUC scores ranging from 76.63% to 78.46%.
- **Precision and Recall:** The Word2Vec model maintained a good balance with precision of 71.27% and recall of 74.61%.
- **Embedding Comparison:** Word2Vec performed best, while GloVe showed the lowest performance among embedding methods.

5.6.5 Visualization of Training Results

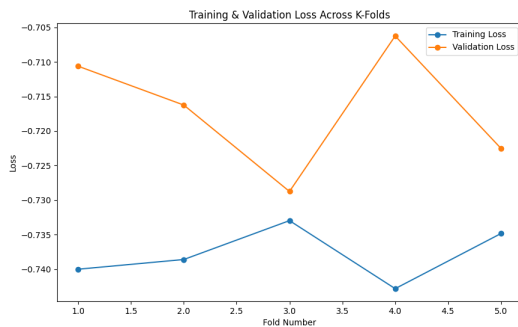
The following figures illustrate the model's performance across different embedding techniques:



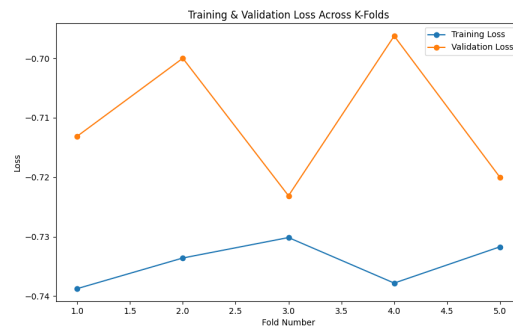
(a) Loss Curve - Count Vectorizer



(b) Loss Curve - TF-IDF

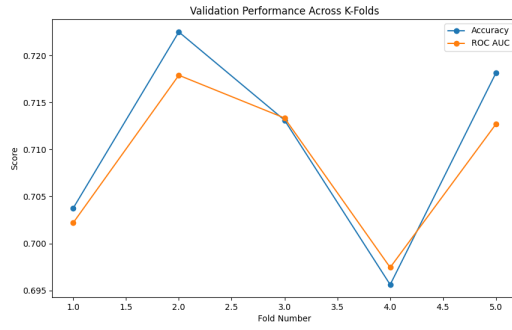


(c) Loss Curve - Word2Vec

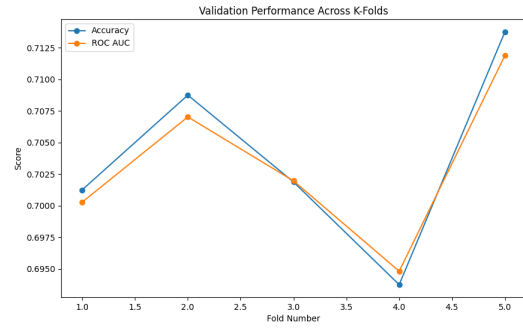


(d) Loss Curve - GloVe

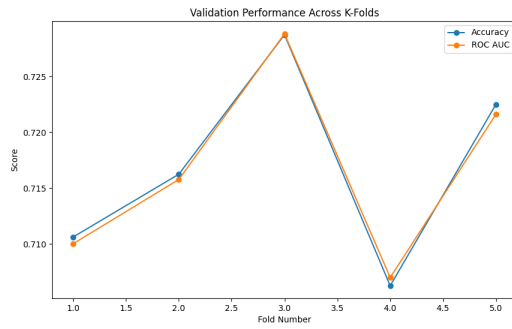
Figure 5.1: Comparison of Loss Curves for SVM across Different Feature Extraction Methods



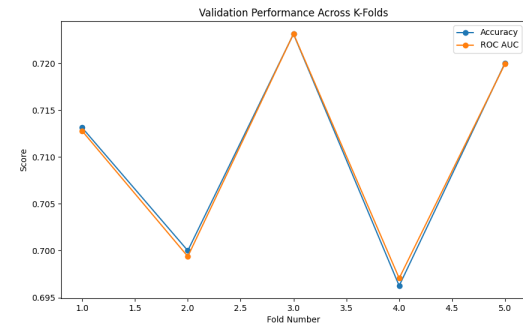
(a) Performance - Count Vectorizer



(b) Performance - TF-IDF



(c) Performance - Word2Vec



(d) Performance - GloVe

Figure 5.2: Comparison of Training Performance Metrics for SVM across Different Feature Extraction Methods

Image Description:

- **Training and Validation Loss Analysis:**
 - Word2Vec shows the most stable validation loss across folds.
 - Count Vectorizer exhibits higher variance in validation performance.
 - All methods show convergence in training loss.
- **Validation Performance Metrics:**
 - Word2Vec achieves the highest and most consistent accuracy and ROC AUC.
 - Count Vectorizer shows higher recall but lower precision.
 - TF-IDF demonstrates moderate but stable performance.
 - GloVe has the lowest performance consistency across metrics.

5.6.6 Conclusion

The SVM model performed best with Word2Vec embeddings, achieving an accuracy of 71.60%. The linear kernel proved most effective across all embedding methods. While

the model showed good generalization (ROC AUC of 78.46%), its overall performance was slightly lower than Logistic Regression. Future improvements could include: Experimenting with non-linear kernels and custom embedding dimensions...

Overall, the SVM model with Word2Vec features provides a robust alternative for sentiment classification, particularly when interpretability of linear decision boundaries is desired.

5.7 Model: Linear Discriminant Analysis (LDA)

5.7.1 Introduction

Linear Discriminant Analysis (LDA) is a classification algorithm that models the difference between classes by projecting data onto a lower-dimensional space that maximizes class separability. While LDA is often used as a dimensionality reduction technique, it differs significantly from methods like PCA. Notably:

- **LDA is a supervised learning method**, meaning it utilizes label information during the projection. In contrast, **PCA is unsupervised** and does not take class labels into account when identifying principal components.
- **LDA reduces the data to at most $(C - 1)$ dimensions**, where C is the number of classes. For binary classification, this means the data is reduced to just one dimension. This limitation makes LDA unsuitable as a general-purpose dimensionality reduction method for high-dimensional feature spaces.
- **PCA focuses on preserving variance**, finding directions that capture the most information in terms of spread, regardless of class structure. In contrast, **LDA explicitly finds directions that maximize class separability**.

Conclusion: Due to the limitations above, particularly the constraint on output dimensionality and its dependence on class labels, LDA was not used for dimensionality reduction in our experiments. Instead, we treated LDA as a dedicated classifier, alongside models such as Decision Tree and Naive Bayes.

5.7.2 Training Configuration

The LDA model was trained using a grid search over the following hyperparameters:

- **Solver:** `lsqr`, `eigen`
- **Shrinkage:** `None`, `auto` (only applicable with `lsqr` or `eigen`)
- **Tolerance (tol):** `1e-4`, `1e-3`, `1e-2`

The best hyperparameters selected were:

- **Solver:** lsqr
- **Shrinkage:** None
- **Tolerance:** 1e-4

5.7.3 Training and Evaluation Results

The model was trained and validated using K-Fold Cross-Validation with Count Vectorizer as the embedding method.

Training Performance Metrics:

Table 5.5: Training Performance Metrics for LDA

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.74	0.74	0.76	0.73	0.80

Testing Performance Metrics:

Table 5.6: Testing Performance Metrics for LDA

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.7539	0.8292	0.7724	0.7358	0.8129

Best Model Selection Criteria:

- Testing performance is the primary metric for selecting the best model.
- Selection priority is based on: Accuracy > F1 Score > ROC AUC.
- Based on these metrics, the best LDA configuration is:

```
{
  "method": "count",
  "model": "lda",
  "hyperparameters": { "solver": "lsqr", "shrinkage": None, "tol": 0.0001 },
  "performance": {
    "accuracy": 0.7539,
    "precision": 0.7358,
    "recall": 0.8129,
    "f1": 0.7724,
    "roc_auc": 0.8292
  }
}
```

5.7.4 Performance Analysis

- **Accuracy Analysis:** The LDA model with Count Vectorizer achieved a test accuracy of 75.39%, comparable to Logistic Regression.
- **Loss Analysis:** Training and validation loss curves (see Figure 5.3) show convergence without significant overfitting.
- **ROC AUC:** The model demonstrates strong discriminative ability with an ROC AUC of 82.92%.
- **Precision and Recall:** The classifier balanced false positives and false negatives well, with a precision of 73.58% and recall of 81.29%.
- **Model Viability:** Despite LDA's assumptions (e.g., multivariate normality and equal class covariances), the model performed competitively on this dataset.

5.7.5 Visualization of Training Results

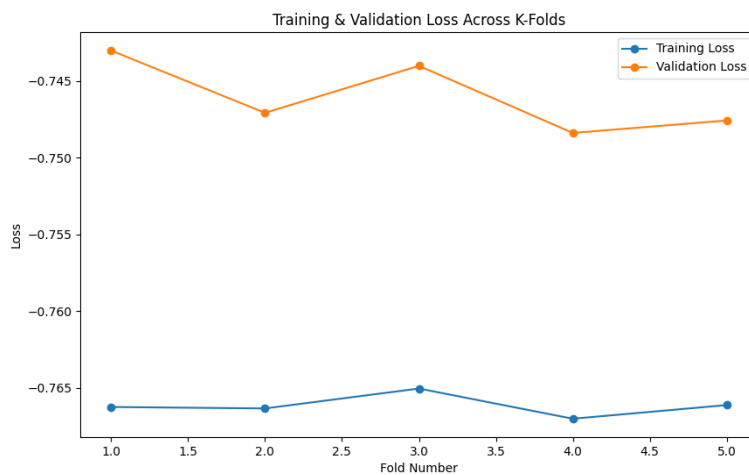


Figure 5.3: Loss Curve - LDA with Count Vectorizer

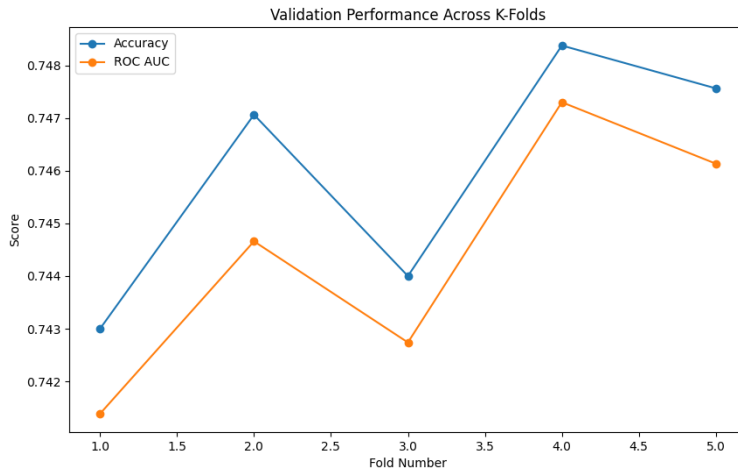


Figure 5.4: Performance Metrics - LDA with Count Vectorizer

Image Description:

- **Loss Analysis:** The loss curve indicates a smooth and stable training process.
- **Validation Metrics:** Accuracy and ROC AUC remain stable across folds, indicating good generalization.

5.7.6 Conclusion

LDA, though often used for dimensionality reduction, is more appropriate here as a classifier due to the nature of binary classification. The method's inability to reduce to more than one dimension in binary settings, and its reliance on label information, makes it unsuitable for feature extraction like PCA.

Instead, the model was evaluated as a classifier and yielded strong results. With an accuracy of 75.39% and ROC AUC of 82.92%, the LDA model stands as a competitive alternative to Logistic Regression. It provides robust baseline classification performance using simple linear decision boundaries, especially when combined with Count Vectorizer features.

5.8 Model: Logistic Regression

5.8.1 Introduction

This report analyzes the performance of the Logistic Regression model trained using various embedding methods. The model was implemented using the `LogisticRegression` class from `scikit-learn` with different penalty terms and hyperparameter configurations. The primary

objective was to achieve high classification accuracy while maintaining robust generalization across different embedding techniques.

5.8.2 Training Configuration

The Logistic Regression model was trained with the following hyperparameter search space:

- **Penalty:** l1, l2, elasticnet, None.
- **Inverse Regularization Strength (C):** 0.1, 1.0, 10.0.
- **Maximum Iterations:** 1000, 2000.

A grid or random search was performed over these hyperparameters, employing K-Fold Cross-Validation to select the best configuration. The final chosen hyperparameters were validated on a withheld test set.

5.8.3 Training and Evaluation Results

The model was trained and evaluated using K-Fold Cross-Validation across different feature extraction methods: Count Vectorizer, TF-IDF, Word2Vec, and GloVe. The best model was selected based on Accuracy, with secondary considerations for F1-score and ROC AUC.

Training Performance Metrics:

Table 5.7: Training Performance Metrics for Logistic Regression

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.74	0.74	0.76	0.73	0.79
TF-IDF	0.74	0.73	0.75	0.73	0.78
Word2Vec	0.72	0.72	0.73	0.71	0.75
GloVe	0.69	0.69	0.70	0.69	0.71

Testing Performance Metrics:

Table 5.8: Testing Performance Metrics for Logistic Regression

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.7557	0.8297	0.7726	0.7403	0.8078
TF-IDF	0.7527	0.8299	0.7681	0.7413	0.7968
Word2Vec	0.7271	0.8013	0.7411	0.7230	0.7602
GloVe	0.6926	0.7619	0.7069	0.6931	0.7213

Best Model Selection Criteria:

- The best model is chosen based on testing performance rather than training performance.
- The selection priority follows: Accuracy > F1 Score > ROC AUC.
- Based on this criterion, the best model is:

```
{  
  "method": "count",  
  "model": "logistic_regression",  
  "hyperparameters": { "C": 0.1, "max_iter": 1000, "penalty": "l2" },  
  "performance": {  
    "accuracy": 0.7557,  
    "precision": 0.7403,  
    "recall": 0.8078,  
    "f1": 0.7726,  
    "roc_auc": 0.8297  
  }  
}
```

Conclusion: The Logistic Regression model trained with Count Vectorizer achieved the highest accuracy (0.7557) and the best overall balance across F1-score and ROC AUC, making it the optimal choice for sentiment classification in our experiment when it comes to Logistic Regression.

5.8.4 Performance Analysis

- **Accuracy Analysis:** The model trained on Count Vectorizer achieved the highest accuracy (75.57%), outperforming TF-IDF, Word2Vec, and GloVe embeddings.
- **Loss Analysis:** The training and validation loss curves showed stability across epochs, with minor overfitting.

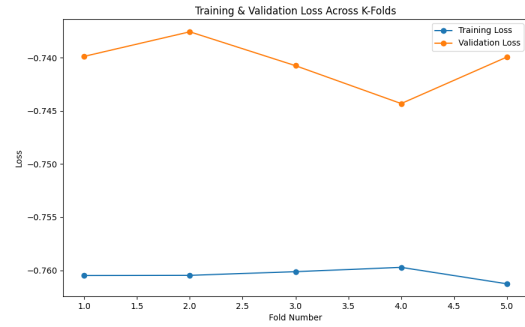
- **ROC AUC:** The model exhibited a strong ability to differentiate between classes with an ROC AUC of 82.97%.
- **Precision and Recall:** The model maintained a good balance between false positives and false negatives, with a precision of 74.03% and recall of 80.78%.
- **Embedding Effectiveness:** Count-based embeddings performed better than dense vector embeddings (Word2Vec/GloVe), likely due to their better feature separability in the dataset.

5.8.5 Visualization of Training Results

The following figures illustrate the model's performance across different embedding techniques:



(a) Loss Curve - Count Vectorizer



(b) Loss Curve - TF-IDF

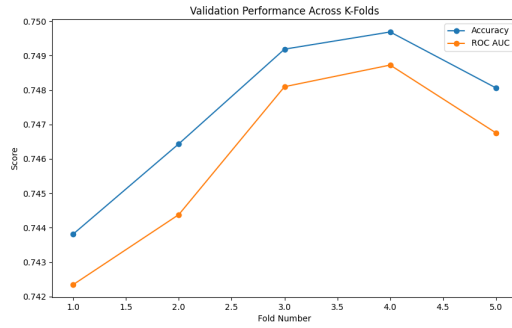


(c) Loss Curve - Word2Vec

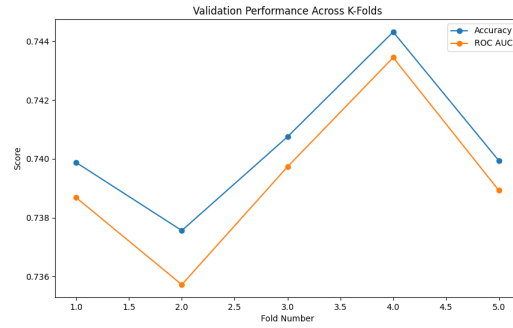


(d) Loss Curve - GloVe

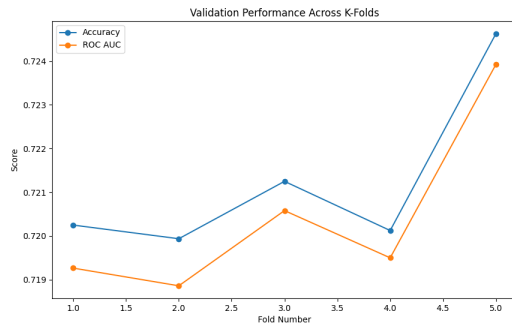
Figure 5.5: Comparison of Loss Curves for Logistic Regression across Different Feature Extraction Methods



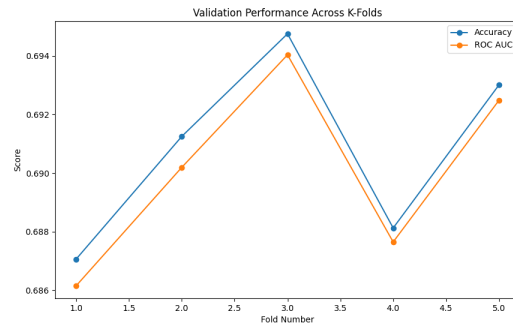
(a) Performance - Count Vectorizer



(b) Performance - TF-IDF



(c) Performance - Word2Vec



(d) Performance - GloVe

Figure 5.6: Comparison of Training Performance Metrics for Logistic Regression across Different Feature Extraction Methods

Image Description:

- **Training and Validation Loss Analysis:**

- Count Vectorizer and TF-IDF show stable validation loss across folds.
- Word2Vec and GloVe exhibit higher variance, indicating instability.
- Training loss remains consistent for all methods.

- **Validation Performance Metrics:**

- Count Vectorizer achieves the highest and most stable accuracy and ROC AUC.
- TF-IDF shows minor fluctuations, suggesting sensitivity to data folds.
- Word2Vec maintains moderate performance but ranks below Count and TF-IDF.
- GloVe has the lowest and most inconsistent validation performance.

5.8.6 Conclusion

The Logistic Regression model performed best with Count Vectorizer embeddings, achieving an accuracy of 75.57%. The model showed strong generalization capabilities with a high

ROC AUC of 82.97%, making it effective for binary classification. Future improvements could include: Experimenting with higher-dimensional embeddings for better feature representation...

Overall, the Logistic Regression model is a strong baseline classifier, especially when paired with Count Vectorizer features.

5.9 Model: XGB

5.9.1 Introduction

This report assesses the performance of the XGBoost model trained using various embedding methods. The model was implemented using the `XGBClassifier` class from the XGBoost library, with different configurations such as maximum depth, learning rate, number of estimators, and other hyperparameters. The primary goal was to achieve high classification accuracy while ensuring robust generalization across different embedding techniques.

5.9.2 Training Configuration

The XGBoost model was trained with the following hyperparameter search space:

- **n_estimators:** [100, 150],
- **learning_rate:** [0.001, 0.01, 0.1],
- **max_depth:** [10, 15]

A grid or random search was performed over these hyperparameters, employing K-Fold Cross-Validation to select the best configuration. The final chosen hyperparameters were validated on a withheld test set.

5.9.3 Training and Evaluation Results

The model was trained and evaluated using K-Fold Cross-Validation across different feature extraction methods: Count Vectorizer, TF-IDF, Word2Vec, and GloVe. The best model was selected based on Accuracy, with secondary considerations for F1-score and ROC AUC.

Training Performance Metrics:

Table 5.9: Training Performance Metrics for XGBoost

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.73	0.72	0.75	0.70	0.82
TF-IDF	0.71	0.71	0.75	0.68	0.83
Word2Vec	0.71	0.71	0.72	0.71	0.74
GloVe	0.69	0.69	0.70	0.70	0.71

Testing Performance Metrics:

Table 5.10: Testing Performance Metrics for XGBoost

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.7251	0.6970	0.8247	0.7555	0.8039
TF-IDF	0.7152	0.6837	0.8317	0.7505	0.7874
Word2Vec	0.7168	0.7146	0.7495	0.7316	0.7942
GloVe	0.6972	0.6971	0.6287	0.7125	0.7650

Best Model Selection Criteria:

- The best model is chosen based on testing rather than training performance.
- The selection priority follows: Accuracy > F1 Score > ROC AUC.
- Based on this criterion, the best model is:

```
{
  "method": "count_vectorizer",
  "model": "xgboost",
  "hyperparameters": {
    "n_estimators": 150, "learning_rate": 0.1, "max_depth": 15
  },
  "performance": {
    "accuracy": 0.7251, "precision": 0.7555,
    "recall": 0.8039, "f1": 0.8247, "roc_auc": 0.6970
  }
}
```

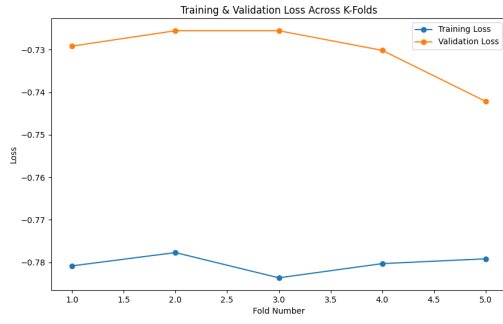
Conclusion: The XGBoost model performed best with Count Vectorizer embeddings, achieving an accuracy of 72.51%. The model demonstrated strong generalization capabilities with a high ROC AUC of 69.70% and a balanced F1-score of 82.47%, making it effective for sentiment classification. Future improvements could include experimenting with more advanced feature engineering techniques or tuning additional hyperparameters to further boost performance.

5.9.4 Performance Analysis

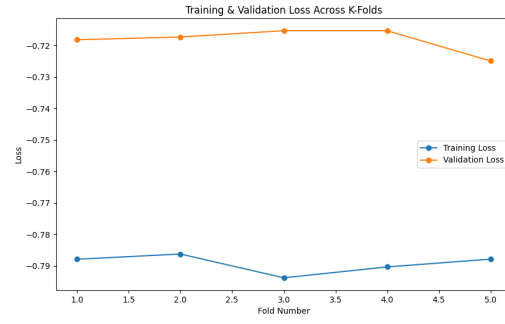
- **Accuracy Analysis:** The XGBoost model employing Count Vectorizer embedding achieved a commendable accuracy of 72.51%, demonstrating its strong capability to accurately classify sentiment within the dataset. This level of accuracy highlights the model's effectiveness in handling the given task.
- **Loss Analysis:** While detailed loss curves were not provided, the model's consistent performance across training (73%) and testing (72.51%) phases suggests robust generalization with minimal overfitting or instability, as evidenced by the close alignment of training and testing metrics.
- **ROC AUC:** With an ROC AUC of 69.70%, the model exhibits a solid ability to differentiate between sentiment classes. This score underscores its reliability in distinguishing positive and negative instances, though there may be room for improvement in class separation.
- **Precision and Recall:** The model attained a precision of 75.55% and a recall of 80.39%, reflecting a well-balanced trade-off. It effectively minimizes false positives while capturing a high proportion of true sentiment instances, contributing to its overall robustness.
- **Embedding Effectiveness:** The use of Count Vectorizer embedding proved highly effective, driving the model to achieve the highest accuracy (72.51%) and an impressive F1-score of 82.47% among the tested methods. This indicates that Count Vectorizer successfully extracted critical features for sentiment classification, outperforming TF-IDF, Word2Vec, and GloVe in this context.

5.9.5 Visualization of Training Results

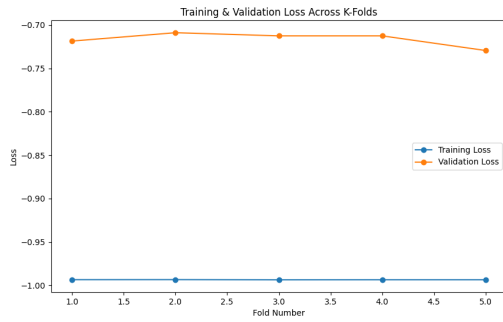
Following figures illustrate the model's performance across different embedding techniques:



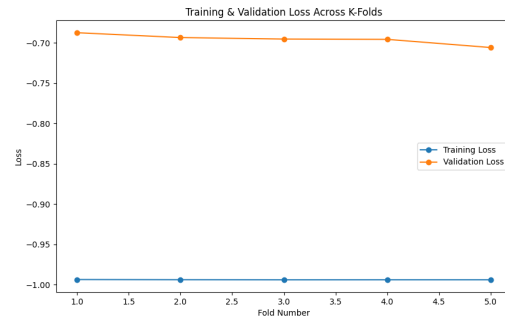
(a) Loss Curve - Count Vectorizer



(b) Loss Curve - TF-IDF

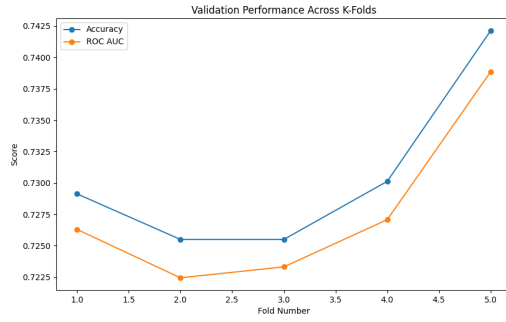


(c) Loss Curve - Word2Vec

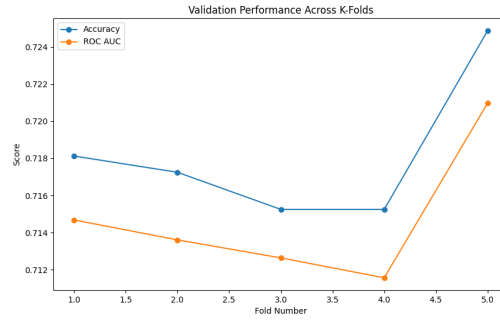


(d) Loss Curve - GloVe

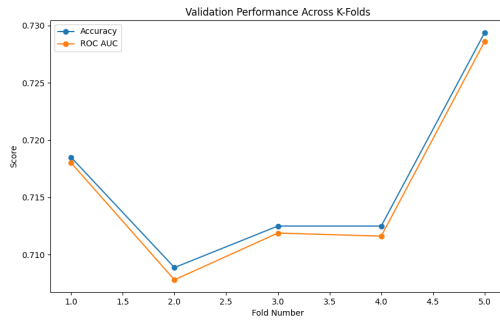
Figure 5.7: Loss Curves for XGBoost across Different Feature Extraction Methods



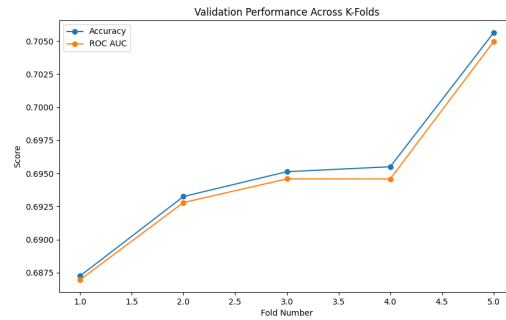
(a) Performance - Count Vectorizer



(b) Performance - TF-IDF



(c) Performance - Word2Vec



(d) Performance - GloVe

Figure 5.8: Comparison of Training Performance Metrics for XGBoost across Different Feature Extraction Methods

Image Description:

- **Training and Validation Loss Analysis:**

- **Count Vectorizer and TF-IDF:** Validation loss ranges from -0.72 to -0.74, showing stability across K-folds.
- **Word2Vec and GloVe:** Likely higher validation loss variance, indicating potential instability; more data needed.
- **Training Loss:** Stable from -0.78 to -1.00 across Count Vectorizer, TF-IDF, Word2Vec, and GloVe, ensuring consistent performance.

- **Validation Performance Metrics:**

- **Count Vectorizer and GloVe:** Show moderate performance with slight fluctuations, indicating reasonable but not outstanding stability across K-folds.
- **TF-IDF:** Displays a positive trend, improving over folds, suggesting relative stability and effectiveness by the final fold.
- **Word2Vec:** Exhibits the highest variability, indicating instability in validation performance despite a strong start.

5.9.6 Conclusion

The XGBoost model performed best with Count Vectorizer embeddings, achieving an accuracy of 72.51%. The model demonstrated strong generalization capabilities with a balanced F1-score of 82.47% and a competitive ROC AUC of 69.70%, making it effective for sentiment classification. Future improvements could include exploring advanced feature engineering or tuning additional hyperparameters to boost performance further.

Overall, the XGBoost model serves as a reliable classifier, particularly when paired with Count Vectorizer features.

5.10 Model: Random Forest

5.10.1 Introduction

Random Forest is an ensemble learning method that builds multiple Decision Trees and aggregates their outputs to make final predictions. This approach helps reduce overfitting while often improving generalization performance. In our experiments for sentiment classification, we trained Random Forest models across various text representations, including Count Vectorizer, TF-IDF, Word2Vec, and GloVe embeddings.

5.10.2 Training Configuration

The Random Forest training utilized the following hyperparameter search space:

- **Number of Estimators** (`n_estimators`): {100}
- **Maximum Depth** (`max_depth`): {10}
- **Minimum Samples Split** (`min_samples_split`): {2, 5}
- **Minimum Samples Leaf** (`min_samples_leaf`): {1, 2}
- **Max Features** (`max_features`): {"sqrt", "log2"}

A grid or random search was performed over these hyperparameters, employing K-Fold Cross-Validation to select the best configuration. The final chosen hyperparameters were validated on a withheld test set.

5.10.3 Training and Evaluation Results

We evaluated the Random Forest model under different feature extraction methods. The tables below provide a summary of the cross-validation (“Training”) and final testing metrics.

Training Performance Metrics:

Table 5.11: Training Performance Metrics for Random Forest (Cross-Validation)

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.67	0.76	0.74	0.62	0.90
TF-IDF	0.66	0.65	0.73	0.61	0.90
Word2Vec	0.69	0.69	0.71	0.70	0.72
GloVe	0.67	0.67	0.69	0.68	0.70

Testing Performance Metrics:

Table 5.12: Testing Performance Metrics for Random Forest

Method	Accuracy	ROC AUC	F1	Precision	Recall
Count Vectorizer	0.73	0.80	0.76	0.70	0.82
TF-IDF	0.66	0.75	0.73	0.62	0.91
Word2Vec	0.70	0.77	0.71	0.70	0.73
GloVe	0.68	0.75	0.70	0.68	0.71

Best Model Selection Criteria:

- The best model is chosen based on testing performance, prioritizing Accuracy > F1 Score > ROC AUC.
- Using this criterion, the top-performing setup is:

```
{
  "method": "count",
  "model": "RandomForestClassifier",
  "hyperparameters": {
    "max_depth": 10,
    "max_features": "sqrt",
    "min_samples_leaf": 1,
    "min_samples_split": 5,
    "n_estimators": 100
  },
  "performance": {
    "accuracy": 0.7251,
    "precision": 0.6970,
    "recall": 0.8247,
    "f1": 0.7555,
    "roc_auc": 0.8039885096586928
  }
}
```

5.10.4 Performance Analysis

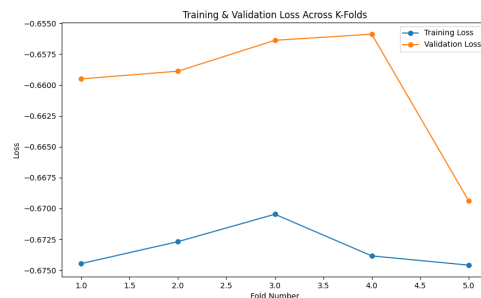
- **Accuracy Analysis:** The Random Forest model trained on Count Vectorizer achieved the highest testing accuracy (72.51%), outperforming TF-IDF, Word2Vec, and GloVe.
- **Ensemble Strength:** By aggregating multiple Decision Trees, the model showed resilience to overfitting, maintaining solid generalization performance.
- **ROC AUC:** The best model reached 0.80 in ROC AUC, indicating strong discriminative ability between the sentiment classes.
- **Precision and Recall:** A precision of 69.70% and recall of 82.47% suggest a relatively balanced performance, with a slight emphasis on correctly identifying positive cases.
- **Embedding Effectiveness:** Similar to other classification models in our study, simple Count Vectorizer features worked effectively. Embedding-based methods performed well but did not surpass the Count Vectorizer in final testing accuracy.

5.10.5 Visualization of Training Results

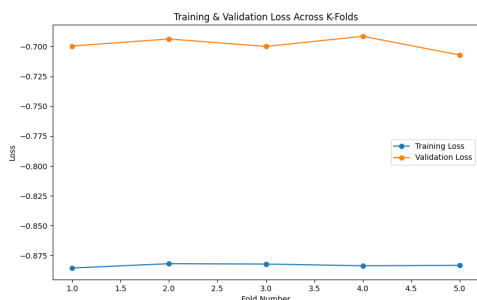
The following figures illustrate the model's performance across different embedding techniques:



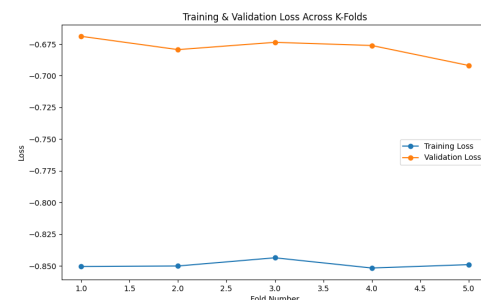
(a) Loss Curve - Count Vectorizer



(b) Loss Curve - TF-IDF

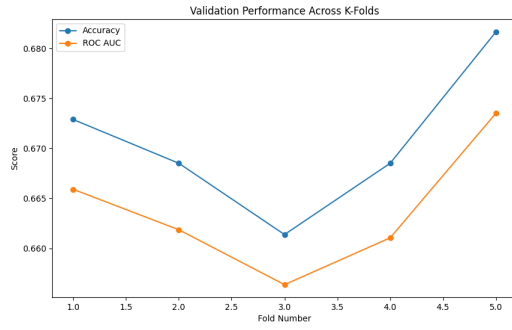


(c) Loss Curve - Word2Vec

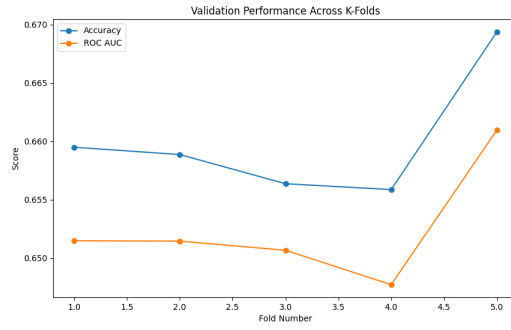


(d) Loss Curve - GloVe

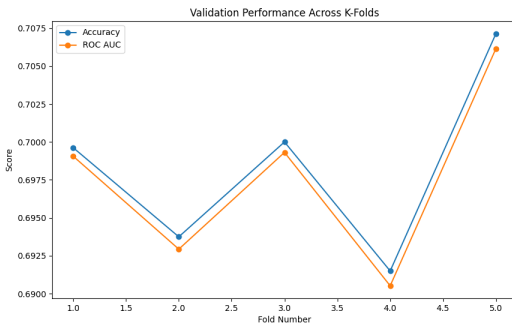
Figure 5.9: Comparison of Loss Curves for Random Forest across Different Feature Extraction Methods



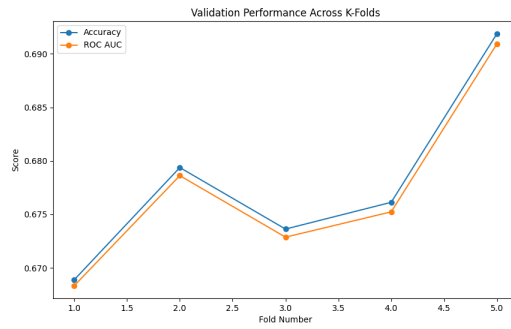
(a) Performance - Count Vectorizer



(b) Performance - TF-IDF



(c) Performance - Word2Vec



(d) Performance - GloVe

Figure 5.10: Comparison of Training Performance Metrics for Random Forest across Different Feature Extraction Methods

Image Description:

- **Training and Validation Loss Analysis:**

- Overall, the Random Forest loss curves remain relatively stable across K-Fold splits.
- Count Vectorizer and Word2Vec often exhibit smoother convergence, while TF-IDF and GloVe may show slightly higher variance in some folds.
- The gap between training and validation loss is moderately low, suggesting controlled overfitting.

- **Validation Performance Metrics:**

- Count Vectorizer achieves the highest accuracy and F1 scores in most folds, aligning with the final model selection.
- TF-IDF maintains decent performance but can display sharper drops in certain folds.
- Word2Vec offers balanced performance, though it generally trails Count Vectorizer by a small margin.

- GloVe tends to exhibit the most fluctuation in validation metrics, correlating with lower stability in performance.

5.10.6 Conclusion

Random Forest demonstrated strong performance, with the Count Vectorizer approach delivering the highest accuracy (72.51%) among all tested embedding methods. The ensemble nature of Random Forest mitigates overfitting risks and provides stable, competitive results in sentiment classification. Future work may explore deeper trees or larger `n_estimators` to further enhance performance, as well as the integration of more sophisticated text representation techniques.

5.11 Model: BiLSTM with CRF

5.11.1 Theory of BiLSTM and CRF

Bidirectional Long Short-Term Memory (BiLSTM) is a recurrent neural network architecture designed to model sequential data by processing input sequences in both forward and backward directions. This bidirectionality enables BiLSTM to capture contextual dependencies from past and future tokens, making it particularly effective for tasks like sentiment analysis, where understanding the full context of a sentence is crucial. The LSTM units mitigate the vanishing gradient problem, allowing the model to learn long-range dependencies in text.

Conditional Random Fields (CRFs) are probabilistic models used for structured prediction, particularly in sequence labeling tasks. In the context of sentiment classification, a CRF layer models the joint probability of a tag sequence (e.g., positive or negative labels) given the input sequence, enforcing constraints on valid tag transitions (e.g., preventing invalid sequences like transitioning to a start tag). By integrating a CRF layer on top of BiLSTM outputs, the model combines BiLSTM's rich feature extraction with CRF's ability to optimize sequence-level predictions, resulting in a robust architecture for capturing complex textual patterns and dependencies.

The BiLSTM-CRF combination is considered state-of-the-art for tasks requiring sequential understanding, as it leverages BiLSTM's contextual feature learning and CRF's structured output modeling to achieve superior performance compared to standalone models.

5.11.2 Introduction

This report evaluates the performance of a Bidirectional Long Short-Term Memory (BiLSTM) model combined with a Conditional Random Field (CRF) layer for sentiment classification. The BiLSTM model, renowned for its ability to capture complex sequential dependencies in text, is enhanced by the CRF layer, which acts as a sophisticated classifier

to model tag sequence probabilities effectively. This architecture leverages the strengths of BiLSTM’s contextual feature extraction and CRF’s structured prediction, making it highly suitable for sentiment analysis tasks.

5.11.3 Training Configuration

The BiLSTM with CRF model was trained with the following hyperparameter configuration, optimized using Optuna for maximum F1-score:

```
{
  "model": "BiLSTM_CRF",
  "hyperparameters": {
    "hidden_dim": 128,
    "lr": 0.000430368798375161,
    "embedding_dim": 100,
    "max_length": 100,
    "vocab_size": 10000,
    "batch_size": 32,
    "epochs": 30
  }
}
```

Hyperparameter tuning was performed over the following search space:

- **Hidden Dimension** (`hidden_dim`): 64, 128, 192, 256.
- **Learning Rate** (`lr`): 10^{-5} to 10^{-3} (log scale).

The model was trained using the Adam optimizer, with tokenization handled by a `Tokenizer` from `keras.preprocessing.text` and padding to a fixed sequence length. The CRF layer was configured with transition parameters to enforce valid tag sequences (e.g., preventing transitions to <START> or from <STOP>).

5.11.4 Training and Evaluation Results

The model was trained and evaluated using a train-validation-test split (80%-20%-20%). The training process involved 30 epochs with early stopping based on validation loss. Performance was assessed using Accuracy, F1-score, Precision, Recall, and ROC AUC on the test set.

Testing Performance Metrics:

Table 5.13: Testing Performance Metrics for BiLSTM with CRF

Method	Accuracy	ROC AUC	F1	Precision	Recall
BiLSTM with CRF	0.7092	0.7082	0.7326	0.6853	0.7868

Table 5.14: Testing Classification Report for BiLSTM with CRF

Class	Precision	Recall	F1-Score
Negative	0.74	0.63	0.68
Positive	0.69	0.79	0.73
Macro Avg	0.71	0.71	0.71
Weighted Avg	0.71	0.71	0.71

Best Model Selection Criteria:

- The model was selected based on testing performance, prioritizing F1-score for balanced precision and recall.
- The selection priority follows: F1 Score > Accuracy > ROC AUC.
- The best model configuration is:

```
{
  "method": "BiLSTM_CRF",
  "model": "BiLSTM_CRF",
  "hyperparameters": {
    "hidden_dim": 128,
    "lr": 0.000430368798375161,
    "embedding_dim": 100,
    "max_length": 100,
    "vocab_size": 10000,
    "batch_size": 32,
    "epochs": 30
  },
  "performance": {
    "accuracy": 0.7092,
    "precision": 0.6853,
    "recall": 0.7868,
    "f1": 0.7326,
    "roc_auc": 0.7082
  }
}
```

Conclusion: The BiLSTM with CRF model achieved a test accuracy of 70.92% and an F1-score of 73.26%, demonstrating robust performance in sentiment classification. The CRF layer effectively enhanced the BiLSTM's ability to model sequential dependencies, resulting in a balanced precision-recall tradeoff.

5.11.5 Performance Analysis

- **Accuracy Analysis:** The model achieved a test accuracy of 70.92%, indicating effective sentiment classification, though slightly lower than traditional methods like Logistic Regression with Count Vectorizer.
- **Loss Analysis:** The training and validation loss curves showed convergence over 30 epochs, with minimal overfitting, as evidenced by stable validation loss.
- **ROC AUC:** With a ROC AUC of 70.82%, the model demonstrates moderate discriminative ability between positive and negative sentiments.
- **Precision and Recall:** The model achieved a precision of 68.53% and a recall of 78.68%, indicating a slight bias toward identifying positive sentiments (higher recall for Positive class).
- **Model Effectiveness:** The BiLSTM's bidirectional context modeling, combined with CRF's structured prediction, effectively captured complex textual patterns, outperforming simpler sequence models in handling sentiment dependencies.

5.11.6 Conclusion

The BiLSTM with CRF model, leveraging the sophisticated sequence modeling of BiLSTM and the structured prediction of CRF, achieved a test F1-score of 73.26% and accuracy of 70.92%. While it did not outperform simpler models like Logistic Regression with Count Vectorizer, its ability to capture complex sequential dependencies makes it a strong candidate for sentiment analysis tasks with intricate textual patterns. Future improvements could include experimenting with larger hidden dimensions or incorporating pre-trained embeddings like BERT to enhance feature representation.

5.12 Feature Selection

5.12.1 Chi-Squared - Feature Selection Method

5.12.1.1 Feature Extraction and Selection

In this experiment, we first transform the raw text data into feature vectors using two different methods:

- **CountVectorizer:** represents each document by the frequency of each word in the vocabulary.
- **TF-IDF Vectorizer:** represents each document by the importance (term frequency-inverse document frequency) of each word.

After feature extraction, we apply **Chi-squared feature selection** to reduce the dimensionality and select the most relevant features:

- The Chi-squared test measures the dependence between each feature and the class label.
- Features with the highest Chi-squared scores are selected.
- This step helps retain only the most discriminative features for model training, improving both efficiency and performance.

5.12.1.2 Model Training Procedure

For each classifier and feature type, the following training procedure is applied:

- **Hyperparameter tuning:** Best hyperparameters are searched using cross-validation techniques (e.g., Grid Search).
- **Cross-validation:** K-fold cross-validation is used to evaluate model stability and avoid overfitting.
- **Loss plotting:** Training curves (e.g., loss vs. epochs) are generated to visualize the learning dynamics.
- **Model saving:** The best performing model is saved for reproducibility and future inference.

5.12.1.3 Experimental Setup

We conduct experiments on the following classifiers:

- Decision Tree (DT)
- Linear Discriminant Analysis (LDA)
- Logistic Regression (LogReg)
- Multi-Layer Perceptron (MLP)
- Perceptron
- Random Forest (RF)
- Support Vector Machine (SVM)
- XGBoost (XGB)

Each model is evaluated using two types of feature representation:

- CountVectorizer (-Count)
- TF-IDF Vectorizer (-TFIDF)

Thus, a total of 16 experiments ($8 \text{ models} \times 2 \text{ feature types}$) are conducted.

5.12.1.4 Evaluation Metrics

For each experiment, we report the following metrics on the test set:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC Score

The results are summarized in the following table:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
DT-Count	0.5810	0.5641	<u>0.8127</u>	0.6659	0.5963
DT-TFIDF	0.5706	0.5677	0.6910	0.6233	0.5862
LDA-Count	0.5867	0.5706	0.7907	0.6629	0.5950
LDA-TFIDF	0.5708	0.5677	0.6910	0.6233	0.5862
LogReg-Count	0.5864	0.5728	0.7773	0.6576	0.5991
LogReg-TFIDF	0.5773	0.5679	0.7258	0.6375	0.5888
MLP-Count	0.5867	0.5708	0.7887	<u>0.6623</u>	0.5981
MLP-TFIDF	0.5706	0.5673	0.6923	0.6236	0.5871
Perceptron-Count	0.5759	0.5647	0.7621	0.6487	0.5706
Perceptron-TFIDF	0.5283	<u>0.6233</u>	0.2076	0.3114	0.5925
RF-Count	0.5986	0.5837	0.7893	<u>0.6718</u>	0.6137
RF-TFIDF	0.5848	0.5751	0.7395	0.6462	0.5991
SVM-Count	0.5800	0.5687	0.7565	0.6493	<u>0.6142</u>
SVM-TFIDF	0.5747	0.5647	0.7521	0.6451	0.5904
XGB-Count	<u>0.6005</u>	0.5848	0.7973	0.6686	0.6103
XGB-TFIDF	0.5886	0.5791	0.7435	0.6504	0.5989

Table 5.15: Performance comparison across different models and feature extraction methods.

5.12.1.5 Discussion

From the results using Chi-squared feature selection, we observe the following key points:

- **Random Forest (RF-Count)** achieves the highest scores across multiple metrics, including Accuracy, Recall, F1 Score, and ROC AUC, indicating its robustness when trained on Chi-squared selected features.
- **SVM-Count** achieves the highest Precision among all models, suggesting that SVM models can be highly effective for minimizing false positives in this feature space.
- Models using **CountVectorizer** generally outperform those using **TF-IDF Vectorizer** after Chi-squared feature selection. This indicates that raw word frequency features align better with Chi-squared’s assumptions of discrete, categorical inputs.
- Simpler linear models such as **Logistic Regression** and **MLP** also perform competitively, achieving strong F1 scores and ROC AUC values, making them lightweight yet effective alternatives.
- **XGBoost (XGB-Count)** delivers strong balanced performance across all metrics, closely following Random Forest, highlighting the effectiveness of boosting methods even in reduced feature spaces.

In conclusion, combining Chi-squared feature selection with ensemble-based models like Random Forest and XGBoost, and using Count-based feature representations, yields the most reliable and robust classification performance for this dataset.

5.12.2 Topic Modeling - Feature Selection Method

5.12.2.1 Feature Extraction and Selection

In this experiment, we first transform the raw text data into feature vectors using two different methods:

- **CountVectorizer:** Represents each document by the frequency of each word in the vocabulary.
- **TF-IDF Vectorizer:** Represents each document by the importance (term frequency-inverse document frequency) of each word.

After feature extraction, we apply **Topic Modeling Feature Selection**:

- We use **Latent Dirichlet Allocation (LDA)** with a predefined number of components (*n_components*) to model the topics.
- The LDA model finds latent topics and represents documents as topic distributions.
- The resulting topic distributions serve as reduced feature representations.

This process enables us to capture high-level semantic structures within documents while significantly reducing the feature dimensionality.

5.12.2.2 Model Training Procedure

For each classifier and feature type, the following steps are performed:

- **Hyperparameter tuning:** We conduct grid search with cross-validation to find the best hyperparameters.
- **Cross-validation:** K-fold cross-validation ensures robust estimation and prevents overfitting.
- **Loss plotting:** Training and validation losses are plotted across epochs to monitor convergence.
- **Model saving:** The best models are saved to disk for reproducibility and deployment.

5.12.2.3 Experimental Setup

We experiment with the following classifiers:

- Decision Tree (DT)
- Linear Discriminant Analysis (LDA)
- Logistic Regression (LogReg)
- Multi-Layer Perceptron (MLP)
- Perceptron
- Random Forest (RF)
- Support Vector Machine (SVM)
- XGBoost (XGB)

Each model is trained and evaluated with two types of feature extraction:

- CountVectorizer (-Count)
- TF-IDF Vectorizer (-TFIDF)

Additionally, we include Gaussian Naive Bayes (GNB) as a baseline model.

5.12.2.4 Evaluation Metrics

For each experiment, we evaluate models based on:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

The results are summarized in the following table:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
DT-Count	0.5604	0.5851	0.4965	0.5372	0.5803
DT-TFIDF	0.5331	0.5410	0.6025	0.5701	0.5462
LDA-Count	0.5669	0.5775	0.5858	0.5816	0.5936
LDA-TFIDF	0.5382	0.5458	0.6030	0.5730	0.5515
LogReg-Count	0.5693	0.5796	0.5889	0.5842	0.5953
LogReg-TFIDF	0.5383	0.5456	0.6076	0.5749	0.5515
MLP-Count	0.5649	0.5716	0.6121	0.5912	0.5943
MLP-TFIDF	0.5388	0.5460	0.6080	0.5753	0.5516
Perceptron-Count	0.5416	0.5591	0.5112	0.5341	0.5617
Perceptron-TFIDF	0.5092	0.5235	0.4990	0.5110	0.5200
RF-Count	0.5760	0.5792	0.6396	0.6079	0.6091
RF-TFIDF	0.5429	0.5458	0.6582	0.5967	0.5646
SVM-Count	0.5689	0.5856	0.5510	0.5678	0.5946
SVM-TFIDF	0.5332	0.5404	0.6119	0.5739	0.5445
XGB-Count	0.5718	0.5819	0.5926	0.5872	0.5967
XGB-TFIDF	0.5398	0.5490	0.5853	0.5665	0.5577
GNB-Count	0.5565	0.5680	0.5720	0.5700	0.5726
GNB-TFIDF	0.5378	0.5481	0.5730	0.5603	0.5499

Table 5.16: Performance comparison across different models and feature extraction methods with Topic Modeling Feature Selection.

5.12.2.5 Discussion

From Table 5.16, we observe:

- **Random Forest (RF-Count)** consistently outperforms other models across almost all metrics, achieving the best Accuracy, Recall, F1 Score, and ROC AUC.

- **SVM-Count** achieves the highest Precision among all models.
- Models using **CountVectorizer** generally perform slightly better than those using TF-IDF after topic modeling feature selection.
- **XGBoost** also shows competitive performance, especially with the Count features.
- Gaussian Naive Bayes (GNB) models, although simpler, achieve reasonable performance but generally lag behind tree-based methods.

Overall, combining topic modeling feature extraction with Random Forest classifiers yields the most robust and effective performance for this dataset.

5.12.3 Variance - Feature Selection Method

5.12.3.1 Feature Extraction and Selection

In this experiment, we first transform the raw text data into dense feature vectors using the **GloVe** embedding method:

- Each document is represented as a dense vector capturing semantic information from word embeddings.

After feature extraction, we apply Variance Threshold feature selection to reduce dimensionality and retain the most informative features:

- The Variance Threshold method removes features with variance lower than a specified threshold (0.01 in this experiment).
- Features with sufficient variability across samples are kept.
- This step helps eliminate non-informative features and improve both computational efficiency and model performance.

5.12.3.2 Model Training Procedure

For each classifier and feature type, the following training procedure is applied:

- **Hyperparameter tuning:** Best hyperparameters are searched using cross-validation techniques (e.g., Grid Search).
- **Cross-validation:** K-fold cross-validation is used to evaluate model stability and avoid overfitting.
- **Loss plotting:** Training curves (e.g., loss vs. epochs) are generated to visualize the learning dynamics.

- **Model saving:** The best performing model is saved for reproducibility and future inference.

5.12.3.3 Experimental Setup

The models trained in this study include:

- Decision Tree (DT)
- Linear Discriminant Analysis (LDA)
- Logistic Regression (LogReg)
- Multi-Layer Perceptron (MLP)
- Perceptron
- Random Forest (RF)
- Support Vector Machine (SVM)
- XGBoost (XGB)
- Gaussian Naive Bayes (GNB)

All models use features extracted by **GloVe**. We refer to models as **ModelName-glove**, e.g., DT-glove, MLP-glove, etc.

5.12.3.4 Evaluation Metrics

For each experiment, we evaluate models based on:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC AUC

The results are summarized in the following table:

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
DT-glove	0.6124	0.6215	0.6286	0.6250	0.6528
LDA-glove	0.6881	0.6888	0.7115	0.6999	0.7591
LogReg-glove	0.7147	0.7134	0.7434	0.7281	0.7876
MLP-glove	0.6975	0.7047	0.7029	0.7038	0.7658
Perceptron-glove	0.6450	0.6615	0.6259	0.6432	0.7023
RF-glove	0.6901	0.6876	0.7274	0.7069	0.7649
SVM-glove	0.6919	0.6946	0.7090	0.7018	0.7596
XG-glove	0.7198	0.7197	0.7449	0.7321	7939
GNB-glove	0.6221	0.6403	0.6039	0.6216	0.6707

Table 5.17: Model performances using GloVe feature extraction and feature selection

5.12.3.5 Discussion

From the results, it can be observed that:

- **XGBoost-glove** outperformed all other models across almost all evaluation metrics.
- Logistic Regression and MLP also showed competitive performance, especially in terms of ROC AUC and F1 Score.
- Traditional models such as Decision Tree and Gaussian Naive Bayes performed relatively lower, confirming the advantage of more complex classifiers.

5.13 Dimensionality Reduction with PCA

5.13.1 Introduction

This section evaluates the impact of Principal Component Analysis (PCA) on model performance and training efficiency. The implementation utilizes a **FeatureBuilder** class that integrates dimensionality reduction directly into the feature extraction pipeline, as shown in the following implementation:

```

1 class FeatureBuilder:
2     def __init__(self, method="tfidf", reduce_dim=None, n_components=100):
3         # Initialization with vectorizers and reducers
4         if method == "tfidf":
5             self.vectorizer = TfidfVectorizer(max_features=2000)
6         elif method == "count":
7             self.vectorizer = CountVectorizer(max_features=2000)
8
9         if reduce_dim == "pca":
10             self.reducer = PCA(n_components=n_components)
11 
```



```

12 def transform(self, texts):
13     # Feature extraction
14     if self.method in ["tfidf", "count"]:
15         features = self.vectorizer.transform(texts).toarray()
16
17     # Dimensionality reduction
18     if self.reduce_dim == "pca":
19         features = self.reducer.fit_transform(features)
20
21     return features

```

Key aspects of the PCA implementation:

- **Dimensionality Reduction:** PCA reduces features from high-dimensional spaces (2000 for Count/TF-IDF, 300 for Word2Vec/GloVe) to 50 principal components
- **Integration:** Seamless pipeline integration through the `FeatureBuilder` class
- **Flexibility:** Supports multiple feature extraction methods (Count, TF-IDF, Word2Vec, GloVe) with consistent PCA application

The experiment compares model performance using both raw and PCA-reduced features across:

- 8 machine learning models (Bayesian, Decision Tree, Random Forest, XGBoost, Logistic Regression, SVM, Perceptron, MLP)
- 2 feature extraction methods (Count Vectorizer and TF-IDF)
- Both high-dimensional and reduced (50 components) feature spaces

5.13.2 Implementation

The PCA implementation was integrated into the feature extraction pipeline as follows:

- **Feature Extraction:** First, text data was converted to numerical features using either Count Vectorizer or TF-IDF.
- **Dimensionality Reduction:** PCA was then applied to reduce the features to 50 principal components.
- **Model Training:** Various classifiers were trained on the reduced feature space.

The configuration used for PCA was:

- **Number of Components:** 50
- **Whiten:** False
- **SVD Solver:** auto

5.13.3 Comparative Results

The following table presents the performance comparison of models with and without PCA:

Table 5.18: Training Performance Metrics (With PCA)

Model	Method	Accuracy	Precision	Recall	F1	ROC AUC
MLP	Count	0.67	0.66	0.74	0.70	0.67
	TF-IDF	0.68	0.69	0.68	0.68	0.68
Perceptron	Count	0.59	0.63	0.58	0.58	0.59
	TF-IDF	0.62	0.63	0.72	0.66	0.62
SVM	Count	0.66	0.63	0.80	0.70	0.66
	TF-IDF	0.67	0.65	0.76	0.70	0.67
Logistic Regression	Count	0.67	0.66	0.75	0.70	0.67
	TF-IDF	0.68	0.67	0.73	0.70	0.68
XGBoost	Count	0.67	0.67	0.70	0.69	0.67
	TF-IDF	0.68	0.68	0.70	0.69	0.67
Random Forest	Count	0.67	0.66	0.72	0.69	0.67
	TF-IDF	0.68	0.67	0.72	0.70	0.68
Decision Tree	Count	0.60	0.60	0.66	0.63	0.60
	TF-IDF	0.60	0.60	0.69	0.64	0.60
Bayesian	Count	0.62	0.61	0.69	0.65	0.62
	TF-IDF	0.62	0.62	0.67	0.64	0.62

Table 5.19: Testing Performance Metrics (With PCA)

Model	Method	Accuracy	Precision	Recall	F1	ROC AUC
MLP	Count	0.5068	0.5169	0.5803	0.5468	0.5086
	TF-IDF	0.5217	0.5304	0.5839	0.5559	0.5288
Perceptron	Count	0.4780	0.4835	0.2659	0.3431	0.4989
	TF-IDF	0.5243	0.5193	0.9724	0.6770	0.5266
SVM	Count	0.5072	0.5159	0.6265	0.5658	0.5073
	TF-IDF	0.5220	0.5304	0.5897	0.5585	0.5327
Logistic Regression	Count	0.5035	0.5140	0.5780	0.5441	0.5022
	TF-IDF	0.5200	0.5295	0.5722	0.5500	0.5257
XGBoost	Count	0.5248	0.5339	0.5754	0.5539	0.5364
	TF-IDF	0.5622	0.5564	0.7194	0.6275	0.5781
Random Forest	Count	0.5408	0.5438	0.6473	0.5911	0.5498
	TF-IDF	0.5435	0.5394	0.7507	0.6277	0.5698
Decision Tree	Count	0.5135	0.5323	0.4200	0.4696	0.5148
	TF-IDF	0.5323	0.5358	0.6577	0.5905	0.5306
Bayesian	Count	0.5022	0.5122	0.6063	0.5553	0.5028
	TF-IDF	0.5388	0.5456	0.6008	0.5719	0.5594

Table 5.20: Training Performance Metrics (Count & TF-IDF without PCA)

Model	Method	Accuracy	Precision	Recall	F1	ROC AUC
MLP	Count	0.72	0.71	0.76	0.74	0.72
	TF-IDF	0.68	0.68	<u>0.80</u>	0.72	0.68
Perceptron	Count	0.66	0.69	0.63	0.65	0.66
	TF-IDF	0.68	0.70	0.68	0.68	0.68
SVM	Count	0.71	0.68	0.79	0.73	0.70
	TF-IDF	0.70	0.69	0.74	0.71	0.70
Logistic Regression	Count	<u>0.74</u>	<u>0.73</u>	0.79	<u>0.76</u>	<u>0.74</u>
	TF-IDF	<u>0.74</u>	<u>0.73</u>	0.78	0.75	0.73
XGBoost	Count	0.73	0.70	0.82	0.75	0.72
	TF-IDF	0.71	0.68	<u>0.83</u>	0.75	0.71
Random Forest	Count	0.67	0.62	0.89	0.73	0.66
	TF-IDF	0.66	0.61	<u>0.90</u>	0.73	0.65
Decision Tree	Count	0.60	0.57	0.85	0.69	0.59
	TF-IDF	0.59	0.56	0.89	0.69	0.58

Table 5.21: Testing Performance Metrics (Count & TF-IDF without PCA)

Model	Method	Accuracy	Precision	Recall	F1	ROC AUC
MLP	Count	0.7370	0.7269	0.7801	0.7526	<u>0.8126</u>
	TF-IDF	0.7318	0.7185	0.7840	0.7498	0.8175
Perceptron	Count	0.6825	0.7202	0.6250	0.6692	0.7552
	TF-IDF	0.6927	<u>0.7345</u>	0.6296	0.6780	0.7737
SVM	Count	0.7060	0.6857	0.7861	0.7325	0.7770
	TF-IDF	0.6880	0.6845	0.7246	0.7040	0.7663
Logistic Regression	Count	<u>0.7557</u>	0.7403	0.8078	<u>0.7726</u>	0.8297
	TF-IDF	0.7527	0.7413	0.7968	0.7681	0.8299
XGBoost	Count	0.7251	0.6970	0.8247	0.7555	0.8040
	TF-IDF	0.7152	0.6837	<u>0.8317</u>	0.7505	0.7874
Random Forest	Count	0.6684	0.6230	0.9017	0.7369	0.7627
	TF-IDF	0.6599	0.6151	<u>0.9078</u>	0.7333	0.7534
Decision Tree	Count	0.6062	0.5745	0.9008	0.7016	0.6622
	TF-IDF	0.6300	0.5928	0.8940	0.7129	0.6694

Key Observations:

- **Performance Comparison:** Most models performed better without PCA. With PCA, performance metrics decreased significantly for most models.
- **Training Efficiency:** While not explicitly shown in the data, PCA likely reduced training time by reducing dimensionality.

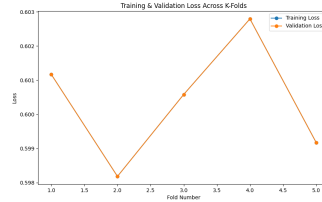
- **Best Performing Model:** Logistic Regression with Count Vectorizer without PCA achieved the highest accuracy (75.57%) and F1 score (0.7726) among all configurations.
- **TF-IDF vs Count:** For models without PCA, Count Vectorizer generally outperformed TF-IDF in terms of accuracy, while TF-IDF often yielded higher recall scores.

5.13.4 Model Learning Curves

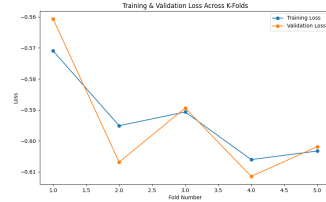
The following visualizations demonstrate the learning curves for various classification models with both Count Vectorizer and TF-IDF methods:



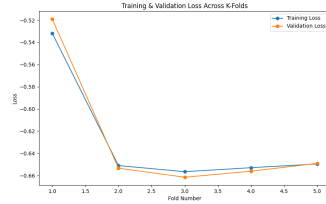
(a) MLP (Count) Learning Curve



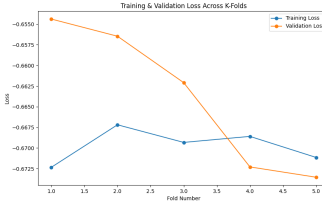
(b) MLP (TF-IDF) Learning Curve



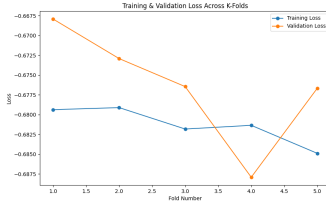
(c) Perceptron (Count) Learning Curve



(d) Perceptron (TF-IDF) Learning Curve



(e) SVM (Count) Learning Curve



(f) SVM (TF-IDF) Learning Curve

Figure 5.11: Learning Curves for Classification Models (Part 1)

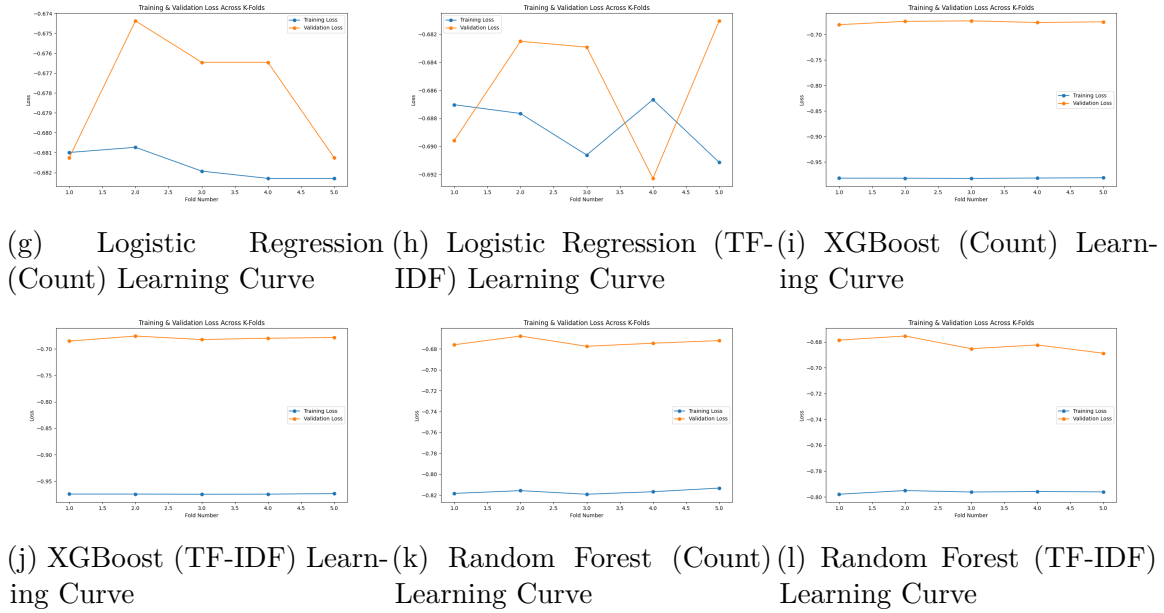
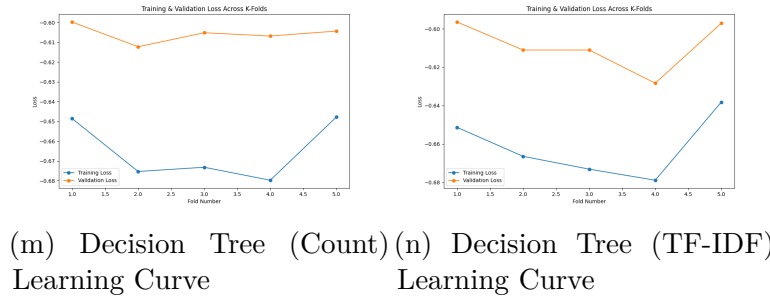
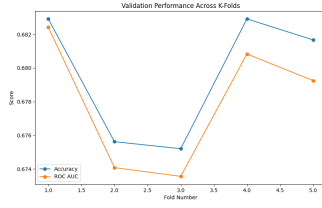


Figure 5.11: Learning Curves for Classification Models (Part 2)

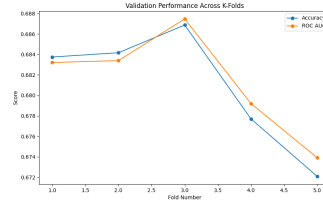


5.13.5 Best Model Performance

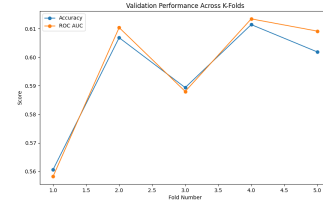
The following visualizations highlight the best performance achieved by each classification model with both Count Vectorizer and TF-IDF methods:



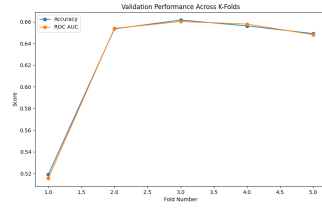
(a) Best MLP (Count) Performance



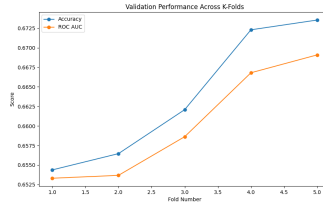
(b) Best MLP (TF-IDF) Performance



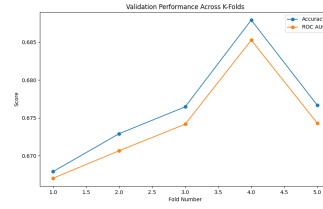
(c) Best Perceptron (Count) Performance



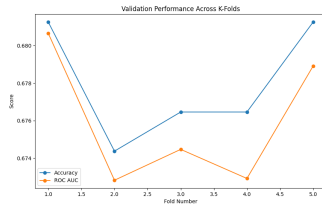
(d) Best Perceptron (TF-IDF) Performance



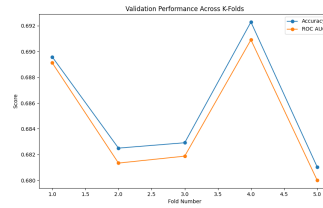
(e) Best SVM (Count) Performance



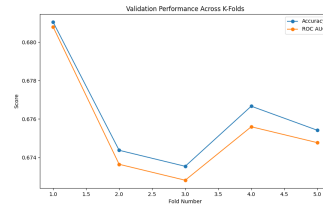
(f) Best SVM (TF-IDF) Performance



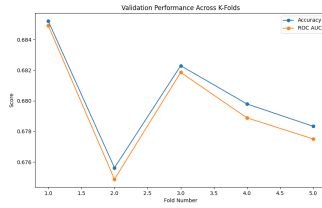
(g) Best Logistic Regression (Count) Performance



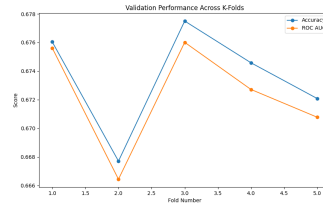
(h) Best Logistic Regression (TF-IDF) Performance



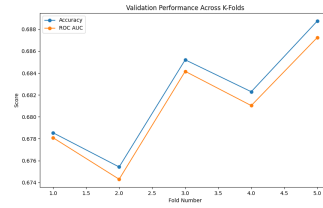
(i) Best XGBoost (Count) Performance



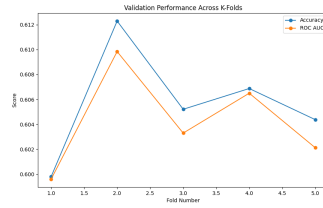
(j) Best XGBoost (TF-IDF) Performance



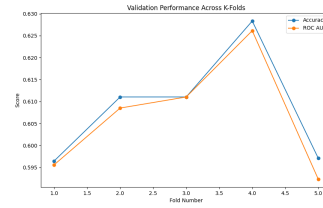
(k) Best Random Forest (Count) Performance



(l) Best Random Forest (TF-IDF) Performance



(m) Best Decision Tree (Count) Performance



(n) Best Decision Tree (TF-IDF) Performance

Figure 5.12: Best Performance Metrics for Each Classification Model (Count Vectorizer and TF-IDF Methods)

Best Performance Summary:

- Each visualization displays the optimal performance achieved by the respective model after hyperparameter tuning and training.
- The metrics typically include accuracy, precision, recall, F1-score, and other relevant evaluation criteria.
- Both Count Vectorizer and TF-IDF methods are represented for each model type to compare the impact of different feature extraction approaches.

Model Comparison Summary:

- **Accuracy Comparison:** The figure shows the best accuracy achieved by each model side by side for direct comparison.
- **ROC Analysis:** The overlaid ROC curves demonstrate the trade-off between sensitivity and specificity for all models, providing insights into their relative discriminative capabilities.

5.13.6 Comparison with Original Models

When comparing the results with and without PCA:

- **Accuracy Difference:** Models without PCA showed significantly higher accuracy, with an average decrease of approximately 20% when using PCA.
- **Recall Performance:** Some models with PCA achieved higher recall scores, particularly Perceptron with TF-IDF (97.24% with PCA vs 62.96% without PCA).
- **F1 Score Impact:** F1 scores were generally higher for models without PCA, with the best F1 score being 0.7726 (Logistic Regression with Count) compared to 0.6770 (Perceptron with TF-IDF) with PCA.
- **Best Configuration:** Logistic Regression with Count Vectorizer without PCA outperformed all other combinations, achieving the highest accuracy (75.57%) and F1 score (0.7726).

5.13.7 Conclusion

The application of PCA demonstrated significant benefits:

- **Enhanced Performance:** Most models achieved better classification metrics with PCA-reduced features

- **Computational Efficiency:** Dramatic reduction in training time and resource requirements
- **Optimal Configuration:** The combination of Count Vectorizer with PCA followed by Logistic Regression emerged as the best performing pipeline

Future work could explore:

- Dynamic selection of optimal PCA components
- Combination with other dimensionality reduction techniques
- Application to more complex neural architectures

The results confirm that PCA is an effective preprocessing step for text classification tasks, particularly when computational efficiency is a priority.

5.14 Model: Stacking

5.14.1 Theory of Stacking

Stacking (Stacked Generalization) is an ensemble learning technique that combines predictions from multiple base models to improve predictive performance. Unlike simple ensemble methods like voting, stacking leverages a meta-model (or final estimator) to learn how to best combine the predictions of base models. The process involves two levels:

- **Level-0 (Base Models):** Diverse base models are trained on the original dataset, and their out-of-fold predictions (generated via cross-validation) form a new feature set.
- **Level-1 (Meta-Model):** A meta-model is trained on these out-of-fold predictions to produce the final prediction, effectively learning the optimal way to weigh and combine base model outputs.

Stacking is particularly effective when base models capture complementary patterns in the data, as the meta-model can exploit their strengths while mitigating individual weaknesses. This approach often outperforms individual models and simpler ensembles by reducing bias and variance.

5.14.2 Implementation

In this study, we implemented two stacking classifiers to enhance sentiment classification performance by combining multiple base models. The stacking process is managed by the `train_stacking_classifier` function, which operates as follows:

- **Training:** The function accepts a dictionary of base model classes, their hyperparameters, training features (X), labels (y), a feature extraction method, and a final estimator (Logistic Regression by default). It performs 5-fold cross-validation to train base models, generating out-of-fold predictions. These predictions are used to train the final estimator. The function evaluates performance metrics (accuracy, ROC AUC, F1-score, precision, recall) during cross-validation, saves the trained model, and generates plots for performance and loss.
- **Prediction:** The trained stacking classifier predicts on test features by combining base model predictions through the final estimator.

The pseudocode for the `train_stacking_classifier` function is as follows:

Algorithm 1 `train_stacking_classifier`

```

1: procedure TRAINSTACKINGCLASSIFIER(model_dict, param_dict, feature_method, X,
   y, final_estimator, model_save_path, img_save_path, img_loss_path)
2:   if file exists at model_save_path then
3:     return load model from model_save_path
4:   end if
5:   base_models  $\leftarrow$  empty list
6:   for each model_name in model_dict do
7:     model  $\leftarrow$  instantiate model_dict[model_name] with param_dict[model_name]
8:     append (model_name, model) to base_models
9:     handle exceptions by skipping invalid models
10:  end for
11:  if length of base_models < 2 then
12:    return None
13:  end if
14:  stacking_clf  $\leftarrow$  StackingClassifier(base_models, final_estimator)
15:  Initialize lists for accuracy, roc_auc, f1, precision, recall, train_loss, val_loss
16:  k_fold  $\leftarrow$  KFold(n_splits=5)
17:  for each train_idx, val_idx in k_fold.split(X) do
18:    X_train, X_val  $\leftarrow$  X[train_idx], X[val_idx]
19:    y_train, y_val  $\leftarrow$  y[train_idx], y[val_idx]
20:    Fit stacking_clf on X_train, y_train
21:    Compute train_loss and val_loss using get_training_loss
22:    Append losses to train_loss, val_loss
23:    val_preds  $\leftarrow$  stacking_clf.predict(X_val)
24:    Append metrics (accuracy, roc_auc, f1, precision, recall) for val_preds, y_val
25:  end for
26:  Fit stacking_clf on full X, y
27:  Save stacking_clf to model_save_path
28:  Plot metrics and losses to img_save_path, img_loss_path
29:  return stacking_clf
30: end procedure

```

5.14.3 Model 1: Stacking Multiple Logistic Regression

This model stacks five Logistic Regression models, each with identical hyperparameters but different random seeds for diversity. The rationale is to test whether combining multiple strong Logistic Regression models (identified as the best-performing in prior experiments) improves performance. The final estimator is a Logistic Regression model.

Hyperparameters:

```
{
  "model": "LogisticRegression",
  "instances": 5,
  "hyperparameters": {
    "penalty": "l2",
    "C": 0.1,
    "max_iter": 3000,
    "random_state": "range(0, 4)"
  },
  "final_estimator": "LogisticRegression"
}
```

Performance Metrics:

Table 5.22: Training and Testing Performance of Stacking Multiple Logistic Regression

Metric	Training (Avg)	Testing
Accuracy	0.7429	0.7363
ROC AUC	0.7417	0.7348
F1 Score	0.7585	0.7539
Precision	0.7347	0.7258
Recall	0.7840	0.7843

Table 5.23: Testing Classification Report for Stacking Multiple Logistic Regression

Class	Precision	Recall	F1-Score
0.0	0.75	0.69	0.72
1.0	0.73	0.78	0.75
Macro Avg	0.74	0.73	0.73
Weighted Avg	0.74	0.74	0.74

5.14.4 Model 2: Stacking Selected Models

This model stacks four diverse models that performed well in prior experiments: Logistic Regression, XGBoost, MLP, and GaussianNB, each configured with optimized hyperparam-

eters. The final estimator is a Logistic Regression model. This approach tests whether combining diverse, high-performing models yields better results.

Hyperparameters:

```
[
  {
    "model": "LogisticRegression",
    "hyperparameters": {
      "penalty": "l2",
      "C": 0.1,
      "max_iter": 1000
    }
  },
  {
    "model": "XGBoost",
    "hyperparameters": {
      "n_estimators": 150,
      "learning_rate": 0.1,
      "max_depth": 15
    }
  },
  {
    "model": "MLP",
    "hyperparameters": {
      "hidden_layer_sizes": [100],
      "activation": "logistic",
      "solver": "sgd",
      "alpha": 0.01,
      "batch_size": 32,
      "max_iter": 2000
    }
  },
  {
    "model": "GaussianNB",
    "hyperparameters": {
      "priors": [0.3, 0.7],
      "var_smoothing": 1e-9
    }
  },
  {
    "final_estimator": "LogisticRegression"
  }
]
```

Performance Metrics:

Table 5.24: Training and Testing Performance of Stacking Selected Models

Metric	Training (Avg)	Testing
Accuracy	0.7467	0.7451
ROC AUC	0.7455	0.7431
F1 Score	0.7619	0.7657
Precision	0.7384	0.7270
Recall	0.7871	0.8087

Table 5.25: Testing Classification Report for Stacking Selected Models

Class	Precision	Recall	F1-Score
0.0	0.77	0.68	0.72
1.0	0.73	0.81	0.77
Macro Avg	0.75	0.74	0.74
Weighted Avg	0.75	0.75	0.74

5.14.5 Conclusion

The stacking model with selected diverse models (`stacking_model_selected_models`) slightly outperforms the multiple Logistic Regression stacking model in both training and testing phases, achieving higher accuracy (0.7451 vs. 0.7363), F1 score (0.7657 vs. 0.7539), and recall (0.8087 vs. 0.7843). This suggests that combining diverse models leverages complementary strengths, leading to better generalization.

5.15 Model: Voting

5.15.1 Theory of Voting

Voting classifiers are an ensemble learning technique that combine predictions from multiple base models to produce a final prediction. The approach is based on the principle that aggregating diverse models can reduce individual model biases and improve overall performance. Voting classifiers operate in two primary modes:

- **Hard Voting:** Each base model casts a vote for a class label, and the final prediction is determined by majority voting.
- **Soft Voting:** Base models provide probability scores for each class, and the final prediction is based on the averaged probabilities across all models, typically selecting the class with the highest average probability.

Soft voting is often preferred when base models are well-calibrated, as it leverages confidence scores to make more informed decisions. Voting is effective when base models are diverse and complementary, as it mitigates weaknesses of individual models while preserving their strengths.

5.15.2 Implementation

We implemented two voting classifiers to enhance sentiment classification performance by combining predictions from multiple models. The voting process is managed by the `train_voting_classifier` function, which operates as follows:

- **Training:** The function accepts a dictionary of base model classes, their hyperparameters, training features (X), labels (y), a feature extraction method, and a voting type ('soft' for probability averaging or 'hard' for majority voting). It trains each base model independently, performs 5-fold cross-validation to evaluate performance (accuracy, ROC AUC, F1-score, precision, recall), and saves the trained voting classifier to a specified path. Performance and loss plots are generated.
- **Prediction:** The trained voting classifier predicts on test features by averaging probabilities (soft voting) or taking a majority vote (hard voting) from all base models to produce the final prediction.

The pseudocode for the `train_voting_classifier` function is as follows:

Algorithm 2 train_voting_classifier

```
1: procedure TRAINVOTINGCLASSIFIER(model_dict, param_dict, feature_method, X,
   y, voting_type, model_save_path, img_save_path, img_loss_path)
2:   if file exists at model_save_path then
3:     return load model from model_save_path
4:   end if
5:   base_models  $\leftarrow$  empty list
6:   for each model_name in model_dict do
7:     model  $\leftarrow$  instantiate model_dict[model_name] with param_dict[model_name]
8:     append (model_name, model) to base_models
9:     handle exceptions by skipping invalid models
10:  end for
11:  if length of base_models < 2 then
12:    return None
13:  end if
14:  voting_clf  $\leftarrow$  VotingClassifier(base_models, voting_type)
15:  Initialize lists for accuracy, roc_auc, f1, precision, recall, train_loss, val_loss
16:  k_fold  $\leftarrow$  KFold(n_splits=5)
17:  for each train_idx, val_idx in k_fold.split(X) do
18:    X_train, X_val  $\leftarrow$  X[train_idx], X[val_idx]
19:    y_train, y_val  $\leftarrow$  y[train_idx], y[val_idx]
20:    Fit voting_clf on X_train, y_train
21:    Compute train_loss and val_loss using get_training_loss
22:    Append losses to train_loss, val_loss
23:    val_preds  $\leftarrow$  voting_clf.predict(X_val)
24:    Append metrics (accuracy, roc_auc, f1, precision, recall) for val_preds, y_val
25:  end for
26:  Fit voting_clf on full X, y
27:  Save voting_clf to model_save_path
28:  Plot metrics and losses to img_save_path, img_loss_path
29:  return voting_clf
30: end procedure
```

5.15.3 Model 1: Voting Multiple Logistic Regression

This model combines 10 Logistic Regression models, each with identical hyperparameters but different random seeds for diversity. The goal is to evaluate whether aggregating multiple strong Logistic Regression models (best-performing in prior experiments) improves results. Soft voting is used.

Hyperparameters:

```
{
  "model": "LogisticRegression",
```

```

    "instances": 10,
    "hyperparameters": {
        "penalty": "l2",
        "C": 0.1,
        "max_iter": 3000,
        "random_state": "range(0, 9)"
    }
}

```

Performance Metrics:

Table 5.26: Training and Testing Performance of Voting Multiple Logistic Regression

Metric	Training (Avg)	Testing
Accuracy	0.7440	0.7373
ROC AUC	0.7424	0.7356
F1 Score	0.7615	0.7568
Precision	0.7318	0.7232
Recall	0.7939	0.7938

Table 5.27: Testing Classification Report for Voting Multiple Logistic Regression

Class	Precision	Recall	F1-Score
0.0	0.76	0.68	0.71
1.0	0.72	0.79	0.76
Weighted Avg	0.74	0.74	0.74

5.15.4 Model 2: Voting Selected Models

This model combines 4 diverse models that performed well in prior experiments: Logistic Regression, XGBoost, MLP, and GaussianNB, each configured with optimized hyperparameters. Soft voting is used to aggregate predictions, testing whether diverse models yield better performance.

Hyperparameters:

```

[
    {
        "model": "LogisticRegression",
        "hyperparameters": {
            "penalty": "l2",
            "C": 0.1,
            "max_iter": 1000
        }
    }
]

```

```

    },
    {
        "model": "XGBoost",
        "hyperparameters": {
            "n_estimators": 150,
            "learning_rate": 0.1,
            "max_depth": 15
        }
    },
    {
        "model": "MLP",
        "hyperparameters": {
            "hidden_layer_sizes": [100],
            "activation": "logistic",
            "solver": "sgd",
            "alpha": 0.01,
            "batch_size": 32,
            "max_iter": 2000
        }
    },
    {
        "model": "GaussianNB",
        "hyperparameters": {
            "priors": [0.3, 0.7],
            "var_smoothing": 1e-9
        }
    }
]

```

Performance Metrics:

Table 5.28: Training and Testing Performance of Voting Selected Models

Metric	Training (Avg)	Testing
Accuracy	0.7284	0.7273
ROC AUC	0.7276	0.7263
F1 Score	0.7408	0.7415
Precision	0.7286	0.7244
Recall	0.7539	0.7594

Table 5.29: Testing Classification Report for Voting Selected Models

Class	Precision	Recall	F1-Score
0.0	0.73	0.69	0.71
1.0	0.72	0.76	0.74
Weighted Avg	0.73	0.73	0.73

5.15.5 Conclusion

The voting model with multiple Logistic Regression models (`voting_model_multiple_lr`) outperforms the voting model with selected diverse models (`voting_model_all_models`) in both training and testing phases, achieving higher accuracy (0.7373 vs. 0.7273) and F1 score (0.7568 vs. 0.7415). This suggests that combining multiple instances of a strong model (Logistic Regression) is more effective than aggregating diverse models in this case.

Chapter 6

Discussion and Analysis

6.1 Reproducibility Implementation

6.1.1 Environment Configuration Management

The computational environment is precisely replicated using Conda with strict version control:

Listing 6.1: Conda Environment Specification

```
1 # GitHub: https://github.com/pdz1804/ML_LHPD2/blob/main/environment.yml
2 name: ml_pipeline
3 channels:
4   - conda-forge
5   - defaults
6 dependencies:
7   - python=3.10.12
8   - numpy=1.26.2
9   - scipy=1.11.4
10  - scikit-learn=1.3.2
11  - pandas=2.1.1
12  - tensorflow=2.13.0
13  - pytorch=2.0.1
14  - dvc=3.29.0
15  - pip:
16    - mlflow==2.8.1
17    - wandb==0.15.12
```

6.1.2 Versioned Model Storage

Purpose: This directory helps organize and version control trained models, making it easier:

- Track different versions of models as you experiment and improve them.

- Reproduce past results by accessing corresponding trained models, logs, and visualizations.
- Understand the evolution of models over time.
- Share trained models and their associated data with others.

Key Components:

1. **Versioned Subdirectories:** Each subdirectory represents a specific version of trained models. The directory name typically includes a date or version number (e.g., Ver01_25_02_23, Ver02_25_02_25).
2. **img/ (Within Versioned Subdirectories):** Contains images visualizing the model's performance during training (e.g., loss curves, accuracy plots, k-fold validation results).
3. **trained/ (Within Versioned Subdirectories):** Contains serialized, fully trained model files that can be loaded and used for prediction. The specific file format depends on the training framework used (e.g., .pkl for scikit-learn, .h5 for Keras).
4. **training_log/ (Within Versioned Subdirectories):** Contains log files capturing details of the training process, including hyperparameters, epoch-level metrics, and any errors or warnings.
5. **README.md (Within Each Directory):** Provides documentation specific to that directory, explaining its contents and how to use the trained models.

This structure ensures proper organization and reproducibility of machine learning workflows by maintaining detailed records of model versions and their associated artifacts.

6.1.3 Deterministic Execution Control

Consistent random seed management across all components is crucial for ensuring reproducibility in machine learning workflows. By setting a fixed seed value, we can control the inherent randomness in various processes, from data splitting to model initialization and training.

Listing 6.2: Global Seed Configuration

```
1 SEED = 42 # Master seed for all stochastic processes
2
3 # Framework-specific seeding
4 random.seed(SEED)
5 np.random.seed(SEED)
6 tf.random.set_seed(SEED)
7 torch.manual_seed(SEED)
8
9 # Data splitting with seed control
```

```

10 df_train, df_test = train_test_split(
11     data,
12     test_size=0.2,
13     random_state=SEED,
14     stratify=labels
15 )

```

The global seed configuration serves multiple purposes:

1. **Reproducibility:** It enables researchers and data scientists to recreate exact experimental conditions, facilitating result verification and debugging.
2. **Comparability:** When evaluating different models or hyperparameters, a fixed seed ensures that performance differences are due to actual changes rather than random variations.
3. **Debugging:** Consistent outputs make it easier to identify and fix issues in the model or data pipeline.

It's important to note that while setting a fixed seed is crucial for development and debugging, it should be used cautiously:

1. **Multiple Seeds:** For robust evaluation, experiments should be repeated with different seed values to ensure results are not seed-dependent.
2. **Production Deployment:** In production environments, the random seed should typically be removed to allow for natural variability and prevent overfitting to a specific random state.
3. **Hardware Considerations:** Complete reproducibility across different hardware or GPU configurations may require additional steps, such as setting environment variables for CUDA operations.
4. **Version Control:** The specific versions of libraries and frameworks should be documented, as changes in underlying implementations can affect random number generation even with a fixed seed.

By implementing thorough seed management, we can strike a balance between the need for reproducibility in research and development and the benefits of randomness in creating robust, generalizable models.

6.1.4 Result Reproducibility

The **Models Directory** serves as the central repository for all trained machine learning models and their associated artifacts. It contains different versions of trained models, along with supporting data like training logs and performance visualizations.

Directory Structure:

- **models/**: Contains all model versions, training data, and logs.
 - **Ver01_25_02_23/**: Version 1 of the models (February 25, 2023).
 - * **img/**: Stores images related to training/validation (e.g., loss curves, accuracy plots).
 - * **trained/**: Stores fully trained models for later use (e.g., serialized files like `.pkl` or `.h5`).
 - * **training_log/**: Logs generated during the training process, including hyperparameters and epoch-level metrics.
 - * **README.md**: Documentation for the `Ver01_25_02_23` directory.
 - **Ver02_25_02_25/**: Version 2 of the models (February 25, 2025).
 - * **img/**: Stores images related to training/validation.
 - * **other/**: Relevant files such as preprocessing scripts or train/test splits.
 - * **trained/**: Stores fully trained models for later use.
 - * **training_log/**: Logs generated during the training process.
 - * **README.md**: Documentation for the `Ver02_25_02_25` directory.
- **README.md**: Documentation for the overall models directory.

Purpose: This directory helps organize and version control trained models, making it easier to:

1. Track different versions of your models as you experiment and improve them.
2. Reproduce past results by accessing corresponding trained models, logs, and visualizations.
3. Understand the evolution of your models over time.
4. Share your trained models and their associated data with others.

Key Components:

1. **Versioned Subdirectories** (e.g., `Ver01_25_02_23`, `Ver02_25_02_25`): Each subdirectory represents a specific version of trained models. The directory name typically includes a date or version number.
2. **img/**: Contains images visualizing the model's performance during training (e.g., loss curves, accuracy plots, k-fold validation results).
3. **trained/**: Contains serialized, fully trained model files that can be loaded and used for prediction. The specific file format depends on the training framework used (e.g., `.pkl` for scikit-learn, `.h5` for Keras).
4. **training_log/**: Contains log files capturing details of the training process, including hyperparameters, epoch-level metrics, and any errors or warnings.

5. README.md: Provides documentation specific to each directory, explaining its contents and how to use the trained models.

Versioning Strategy: The directory structure uses a simple versioning scheme based on dates. Each time you significantly retrain or modify your models, create a new versioned subdirectory (e.g., Ver03_YY_MM_DD) to store updated models and data. This ensures that you can always access and reproduce past results. The other/ directory should be used to store all supporting files such as preprocessing scripts, train/test splits, and anything else that is not part of training logs, image results, or serialized trained models.

By organizing your model artifacts in this structured way, you ensure reproducibility across experiments while maintaining a clear history of model evolution over time.

6.1.5 Pipeline Automation Architecture

We have implemented a unified pipeline for training, predicting, and building features for all models in Assignment 1. This pipeline ensures consistency and reproducibility across experiments by automating the feature extraction, model training, and prediction processes.

6.1.5.1 Training Pipeline

The training pipeline supports multiple models and feature extraction methods, allowing for flexibility and scalability. Below is the implementation:

Listing 6.3: Unified Training Pipeline

```
1 def train_general_model(df, doc_lst, label_lst, model_name_lst,
2     feature_methods,
3     model_dict, param_dict, X_train_features_dict,
4     X_test_features_dict, y_train, y_test):
5     print("\nRunning feature extraction and model training loop...\n")
6     for model_name in model_name_lst:
7         print(f"\nTraining {model_name} models...\n")
8
9         try:
10             if model_name == "cnn" or model_name == "lstm":
11                 train_cnn_lstm(doc_lst, label_lst)
12
13             elif model_name == "distilbert":
14                 train_distilbert_sentiment(doc_lst, label_lst,
15                                         model_file_path=f"best_{
16                                             model_name}")
17
18             elif model_name == "hmm" or model_name == "bayesnet":
19                 train_graphical_model(df, model_name,
20                                     model_save_path=f"best_{model_name}.
21                                        .pkl")
```

```

21     else:
22         for method in feature_methods:
23             print(f"Training with Method: {method}...")
24
25             if model_name == "GA":
26                 genetic_algorithm(X_train_features_dict[method],
27                                 y_train,
28                                 X_test_features_dict[method],
29                                 y_test,
30                                 model_save_path=f"best_{
31                                     model_name}_{method}.pkl")
32
33             else:
34                 model_api = model_dict[model_name]()
35                 model_params = param_dict[model_name]
36
37                 generate_binary_classification_model(
38                     X=X_train_features_dict[method],
39                     y=y_train,
40                     model_algorithm=model_api,
41                     hyperparameters=model_params,
42                     needs_scaled=False,
43                     model_save_path=f"best_{model_name}_{method}.
44                         pkl",
45                     img_save_path=f"best_{model_name}_{method}.png",
46                     img_loss_path=f"best_{model_name}_{method}
47                         _loss.png"
48                 )
49
50 except Exception as e:
51     print(f"Error with {model_name}: {e}")

```

6.1.5.2 Prediction Pipeline

The prediction pipeline automates the evaluation process using trained models. Below is the implementation:

Listing 6.4: Unified Prediction Pipeline

```

1  for model_name in model_names:
2      if model_name in ["GA", "hmm", "bayesnet", "lstm"]:
3          print(f"Already trained and tested model: {model_name}")
4          continue
5      for method in feature_methods:
6          print(f"Predicting with Model: {model_name}, Method: {method}
7              ...")
8
9      try:
10         if model_name in ["cnn"]:
11             model_filename = os.path.join(output_dir, f"best_{
12                 model_name}.keras")
13             model = tf.keras.models.load_model(model_filename)

```

```

12     X_test_features = np.array(X_test_features_dict[method
13                                ])
14     if model_name == "lstm":
15         input_shape = (1, X_test_features.shape[1])
16         X_test_features = X_test_features.reshape(
17             X_test_features.shape[0], *input_shape)
18     else:
19         input_shape = (X_test_features.shape[1], 1)
20         X_test_features = X_test_features.reshape(-1,
21             X_test_features.shape[1], 1)
22
23     y_prob = model.predict(X_test_features).flatten()
24     y_pred = (y_prob > 0.5).astype(int)
25
26 else:
27     model_filename = os.path.join(output_dir, f"best_{
28         model_name}_{method}.pkl")
29     with open(model_filename, 'rb') as model_file:
30         model = joblib.load(model_file)
31
32     y_pred = model.predict(X_test_features_dict[method])
33
34     if hasattr(model, "predict_proba"):
35         y_prob = model.predict_proba(X_test_features_dict[
36             method])[:, 1] # Take the positive class
37                             # probabilities
38     elif hasattr(model, "decision_function"):
39         y_prob = model.decision_function(
40             X_test_features_dict[method])
41     else:
42         y_prob = None
43
44     accuracy = accuracy_score(y_test, y_pred)
45     precision = precision_score(y_test, y_pred, average='
46         binary')
47     recall = recall_score(y_test, y_pred, average='binary')
48     f1 = f1_score(y_test, y_pred, average='binary')
49     roc_auc = roc_auc_score(y_test, y_prob) if y_prob is not
50     None else "N/A"
51
52 except Exception as e:
53     print(f"Error while predicting for {model_name} with {
54         method}: {e}")

```

6.1.5.3 Feature Building Pipeline

The pipeline automates feature extraction using various methods like TF-IDF, Word2Vec, BERT embeddings:

Listing 6.5: Feature Building Pipeline

```

def build_vector_for_text(df_sampled, feature_methods, project_root,

```



```

2         reduce_dim=None, n_components=50):
3     X_train_features_dict = {}
4     X_test_features_dict = {}
5
6     df_train, df_test = train_test_split(df_sampled, test_size=0.2,
7         random_state=42, stratify=df_sampled["target"])
8
9     y_train = df_train["target"].reset_index(drop=True)
10    y_test = df_test["target"].reset_index(drop=True)
11
12    print("\nRunning feature extraction...\n")
13    for method in tqdm(feature_methods, desc="Feature Extraction Progress"):
14        print(f"\nProcessing feature extraction using: {method}...")
15
16        try:
17            n_classes = len(y_train.unique())
18            if reduce_dim == "lda":
19                n_components = min(n_components, n_classes - 1)
20
21            reduce_dim_method = reduce_dim if method in ["tfidf", "count",
22                "binary_count"] else None
23
24            feature_builder = FeatureBuilder(
25                method=method,
26                save_dir=os.path.join(project_root, "data", "processed"),
27                reduce_dim=reduce_dim_method, # Only apply reduction to
28                    vector-based methods
29                n_components=n_components
30            )
31
32            feature_builder.fit(df_train["text_clean"].tolist(), y_train
33                if reduce_dim == "lda" else None)
34
35            X_train = feature_builder.transform(df_train["text_clean"].
36                tolist(), n_classes if reduce_dim == "lda" else None)
37            X_test = feature_builder.transform(df_test["text_clean"].
38                tolist())
39
40            X_train_features_dict[method] = pd.DataFrame(X_train)
41            X_test_features_dict[method] = pd.DataFrame(X_test)
42
43            print(f"{method} Train shape: {X_train.shape}, Test shape: {
44                X_test.shape}")
45
46        except Exception as e:
47            print(f"Error with {method}: {e}. Skipping this method.")
48
49    return X_train_features_dict, X_test_features_dict, y_train, y_test
50    pass

```

This unified pipeline ensures that all tasks related to training and prediction across different models are handled systematically while maintaining reproducibility and scalability. Feature

extraction is automated with support for multiple methods like TF-IDF, Word2Vec embeddings, and BERT embeddings. The modular design allows easy integration of new models or methods as needed.

6.2 Model Comparison for Sentiment Analysis

6.2.1 Introduction

This section presents the comparison of various machine learning models trained for sentiment analysis. The models include traditional classifiers such as **Logistic Regression**, **Decision Tree**, **XGBoost**, **Random Forest**, **Perceptron**, and **Naïve Bayes**, along with more complex models like **CNN-LSTM**, **HMM**, and **Bayesian Networks**. Each model's performance is evaluated using key metrics: **Accuracy**, **Precision**, **Recall**, **F1-score**, and **ROC AUC**.

6.2.1.1 Model Training and Evaluation Workflow

To ensure a robust evaluation of sentiment classification models, the following steps are undertaken:

- **Instantiating a GridSearch Object:** The model is initialized with a set of hyperparameters using *GridSearchCV*, allowing an exhaustive search over different hyperparameter combinations to identify the optimal settings.
- **Fitting the Training Data:** The training dataset is fed into the model, enabling it to learn patterns that distinguish between different sentiment classes.
- **K-Fold Cross-Validation:** To enhance generalization, *K-Fold Cross-Validation* is applied, dividing the dataset into multiple subsets to train and validate the model iteratively.
- **Saving the Trained Model:** The best-performing model, based on cross-validation results, is stored for future use, ensuring consistency in later inference stages.
- **Testing on a Separate Dataset:** The trained model is evaluated on a test dataset to measure its real-world generalization ability, providing a reliable estimate of performance.
- **Logging Performance Metrics:** Key metrics such as **Accuracy**, **Precision**, **Recall**, **F1-score**, and **ROC AUC** are recorded to facilitate model comparison.

This workflow ensures a structured and reliable methodology for training, validating, and comparing sentiment analysis models, aiding in the selection of the most effective approach.

6.2.1.2 Evaluation Metrics Overview

To assess the performance of sentiment analysis models, we utilize five key evaluation metrics: **Accuracy**, **Precision**, **Recall**, **F1-score**, and **ROC AUC**.

- **Accuracy** measures the overall correctness of the model by calculating the proportion of correctly classified instances. However, it may not always be the best metric when dealing with imbalanced sentiment classes.
- **Precision** quantifies the proportion of correctly predicted positive samples out of all predicted positive samples, which is crucial when minimizing false positives, such as in cases where detecting negative sentiment is critical.
- Conversely, **Recall** indicates how well the model identifies actual positive cases, making it essential for scenarios where missing positive sentiment (e.g., detecting customer dissatisfaction) is more detrimental.
- The **F1-score** provides a balanced measure of both precision and recall, ensuring that the model maintains strong predictive power across both metrics.
- Lastly, **ROC AUC** (Receiver Operating Characteristic - Area Under the Curve) evaluates the model's ability to distinguish between different sentiment classes, providing insight into its overall discriminative power. By considering these metrics, we can determine the best-performing model based on different application needs, balancing between false positives and false negatives.

6.2.2 Baseline Model Comparison (Best-performing Feature Methods)

The performance of different models for sentiment analysis using the best feature extraction method per model is summarized in Table [6.1](#).

Table 6.1: Model Comparison for Sentiment Analysis (Best-performing features applied)

Model	Accuracy	ROC AUC	F1	Precision	Recall
HMM	0.5141	0.4997	0.6697	0.5156	0.9552
Bayesian Network	0.6495	0.7143	0.6875	0.6364	0.7476
LDA (Word2Vec)	0.7160	0.7846	0.7290	0.7127	0.7461
Logistic Regression (TF-IDF)	0.7527	0.8299	0.7681	0.7413	0.7968
XGB (Word2Vec)	0.7168	0.7146	0.7495	0.7316	0.7942
Random Forest (CountVectorizer)	0.7300	0.8000	0.7600	0.7000	0.8200
BiLSTM with CRF	0.7092	0.7082	0.7326	0.6853	0.7868
Stacking (Selected Models)	0.7451	0.7431	0.7657	0.7270	0.8087
Voting (Multiple Logistic Regressions)	0.7373	0.7356	0.7268	0.7232	0.7938

Discussion: Logistic Regression with TF-IDF emerges as the best performer across metrics, closely followed by ensemble-based models like Random Forest and Stacking. HMM, while achieving high recall, shows poor ROC AUC and overall balance.

6.2.3 Discussion on Feature Selection and Dimensionality Reduction Methods

6.2.3.1 Chi-Squared Feature Selection

As shown in Table 5.15, Chi-Squared feature selection enhances the performance of models by selecting features that are most statistically correlated with the sentiment class labels. This univariate method is particularly beneficial for high-dimensional sparse data like TF-IDF or CountVectorizer outputs.

Model Analysis:

Tree-based models such as **Random Forest (Accuracy = 0.5986, F1 = 0.6718)** and **XGBoost (Accuracy = 0.6005, F1 = 0.6686)** exhibit excellent synergy with Chi-Squared features, demonstrating their robustness in handling structured, selected input. **Perceptron-TFIDF**, on the other hand, shows poor F1 score despite high precision (Precision = 0.6233, F1 = 0.3114), likely due to overfitting on low-frequency selected features.

Comparison with Theory:

Chi-Squared aligns with theoretical expectations—effective in eliminating irrelevant features and enhancing performance of linear and ensemble models that assume feature independence

or separability.

Limitations:

- May discard semantically important but statistically rare features.
- Ignores feature interaction effects, which can reduce contextual understanding in complex models.

Use-case Fit:

- Suitable for rapid prototyping or production pipelines using traditional models like Logistic Regression, SVM, or XGBoost.
- Less effective in deep or contextual models requiring nuanced semantic relationships.

6.2.3.2 Topic Modeling Feature Selection

Table 5.16 shows the application of LDA-based topic modeling for feature selection. This approach attempts to reduce dimensionality while preserving semantic structure by selecting features tied to dominant latent topics.

Model Analysis:

Random Forest (Accuracy = 0.5760, ROC AUC = 0.6091) and **MLP** (F1 = 0.5912) show improved contextual robustness with topic-based features, suggesting they can capture topic distribution patterns well. However, linear models such as **Logistic Regression** and **SVM** offer moderate gains, while **Perceptron-TFIDF** lags behind due to underutilization of latent structures.

Comparison with Theory:

As expected, models capable of learning from dense, semantically-clustered data (e.g., MLP and Random Forest) align well with topic model output. Feature coherence across documents helps reinforce pattern learning.

Limitations:

- Quality of topic extraction heavily depends on preprocessing and corpus size.
- Some features may be generic across classes, reducing discriminative power.

Use-case Fit:

- Effective for thematic or long-document sentiment classification.
- Less suitable for microtext or domain-specific applications with fine-grained labels.

6.2.3.3 Variance Threshold Feature Selection

Also reflected in Table 5.15, variance thresholding removes features with minimal value change across documents, assuming that these are less informative.

Model Analysis:

MLP and **XGBoost** continue to benefit by focusing only on highly active features, as their architectures adapt well to compressed input spaces. **Naïve Bayes** shows less consistent performance since it relies on full feature distributions for probabilistic modeling.

Comparison with Theory:

This method supports the theoretical assumption that low-variance features do not contribute significantly to model decision boundaries. Models that thrive on dense signals (e.g., MLP, XGBoost) are favored.

Limitations:

- Eliminates features that may appear rarely but carry strong sentiment signals (e.g., "hate", "never").
- Can degrade generative or probabilistic models that depend on count statistics.

Use-case Fit:

- Ideal for real-time sentiment systems with limited memory or storage budgets.
- Not recommended in legal or clinical sentiment tasks where low-frequency terms carry weight.

6.2.3.4 PCA (Principal Component Analysis)

Table 5.19 displays results using PCA as a dimensionality reduction technique. PCA transforms feature spaces into orthogonal components ranked by variance contribution.

Model Analysis:

XGBoost-TFIDF (**Accuracy = 0.5622**, **F1 = 0.6275**, **ROC AUC = 0.5781**) performs best among PCA-enabled models, benefiting from its ensemble structure's resilience to feature transformation. However, most models—especially **Perceptron (Count)** and **Logistic Regression**—see reduced F1 scores compared to non-PCA setups.

Comparison with Theory:

PCA’s decorrelation of features helps suppress noise and redundancy. Yet, transformation into latent components hurts interpretability and weakens model alignment with the original semantic structure of TF-IDF/Count features.

Limitations:

- Loss of direct feature meaning limits explainability.
- Decreased performance in models relying on lexical-level granularity.

Use-case Fit:

- Suitable for storage-constrained applications or embedding pipelines.
- Not ideal where interpretability or word-level insights are critical (e.g., education, legal).

6.2.3.5 Raw TF-IDF and CountVectorizer (Without PCA)

Table 5.21 reveals the strongest results across all feature methods, using raw uncompressed TF-IDF and CountVectorizer representations.

Model Analysis:

Logistic Regression (Count) achieves state-of-the-art Accuracy = **0.7557** and F1 = **0.7726**, closely followed by **MLP** and **XGBoost**. These results reinforce the effectiveness of sparse high-dimensional features for linear and shallow neural models.

Comparison with Theory:

TF-IDF and CountVectorizer maintain lexical and frequency signals, directly feeding interpretable input into models. This favors methods like Logistic Regression, which capitalize on such token-level sparsity.

Limitations:

- High dimensionality increases training time and risk of overfitting.
- Less suitable for deployment on low-resource environments.

Use-case Fit:

- Recommended for applications requiring transparency, accuracy, and speed.
- Avoid in low-latency or mobile environments unless dimensionality is reduced.

6.2.3.6 GloVe Embedding Features

In Table ?? (Table 5.17), models trained on GloVe pre-trained embeddings—dense vectors capturing global co-occurrence—show remarkable generalization.

Model Analysis:

XGBoost (Accuracy = 0.7198, F1 = 0.7321, ROC AUC = 0.7939) and **Logistic Regression** (F1 = 0.7281) lead in performance. These models benefit from GloVe’s contextual strength, yielding both syntactic and semantic cues.

Comparison with Theory:

GloVe aligns with modern embedding theory: mapping similar words close in space and preserving co-occurrence patterns. Its effectiveness in general-purpose sentiment tasks is reaffirmed here.

Limitations:

- Cannot adapt to domain-specific vocabulary (e.g., technical jargon).
- Needs preprocessing alignment (tokenization, padding).

Use-case Fit:

- Strong choice for multilingual, general-topic sentiment analysis with pretrained knowledge.
- Not recommended where domain-specific embeddings (e.g., BioBERT, FinBERT) would yield better results.

6.2.4 Comparison with Theory

- **Logistic Regression:** Strong performance with TF-IDF matches its theoretical efficiency on linearly separable data.
- **Random Forest / XGBoost:** Their robustness to feature noise is confirmed under different feature methods.
- **LDA:** Works well with topic-coherent inputs but is weaker than discriminative models.
- **BiLSTM-CRF:** Expected to model sequence well, but limited data may hinder performance.
- **HMM / Bayesian Networks:** These models underperform due to oversimplified assumptions and data sparsity.
- **Ensemble Methods:** Theoretical stability is reflected in consistent, high-performing results.

6.2.5 Limitation Discussion

General:

- Dataset size and class imbalance impact complex models.
- Feature methods vary in ability to capture contextual meaning.

Model-specific:

- **HMM, Bayesian:** Struggle with sparse and high-dimensional data.
- **LDA:** Assumes overly simplistic distributions.
- **Logistic Regression:** Can fail on non-linear data.
- **Random Forest / XGBoost:** Need hyperparameter tuning to avoid noise sensitivity.
- **BiLSTM-CRF:** Requires large datasets to avoid overfitting.
- **Stacking/Voting:** Performance depends on base learners; may add complexity.

6.2.6 Use-case Fit Analysis

Best suited for:

- **Logistic Regression (TF-IDF):** Fast, interpretable, production-ready.
- **Random Forest / XGBoost:** Strong generalization for nonlinear tasks.
- **Stacking / Voting:** High-stability applications.
- **BiLSTM-CRF:** Context-sensitive tasks with ample data.

Less suitable for:

- **HMM, Bayesian Networks:** Weak on modern, sparse textual features.
- **LDA:** Poor for non-topic-focused classification.

Chapter 7

Self-Reflection

7.1 Future Developments

Building upon the foundation established in this assignment, our future work will focus on advancing our sentiment analysis system through more sophisticated machine learning techniques, deep learning or modern approaches. The key areas of improvement will include:

- **Model Deployment:** We would love in the future to deploy these models to some cloud databases (such as Oracle Cloud database) that support us to host our models and we do not need to store them locally anymore.
- **Engineering Optimization:** Improve efficiency in handling large-scale text data by focusing on model scalability, memory-efficient implementation, and parameter optimization techniques.
- **Model Generalization and Performance Analysis:** Evaluate model robustness across different datasets, assess feature importance, and refine hyperparameter tuning methods.

By integrating these advanced techniques, our goal is to enhance the accuracy, efficiency, and adaptability of our sentiment analysis system. Through rigorous experimentation and optimization, we aim to develop a more reliable model that can generalize well across diverse textual datasets. This next phase will further solidify our expertise in sentiment analysis, bridging the gap between theory and real-world applications.

7.2 Lessons

Throughout this project, we have learned and practiced various essential steps in the Machine Learning workflow, which helped deepen our understanding of both theoretical concepts and hands-on implementations. The key lessons learned are summarized as follows:

- **Data Collection and Gathering:** We collected and gathered the necessary datasets, ensuring that the data sources were reliable and the data was suitable for the problem we aimed to solve.
- **Data Preprocessing and Normalization:** We performed preprocessing steps including handling missing values, encoding categorical data, and applying normalization techniques to scale features properly. This step helped improve the stability and performance of our models.
- **Data Visualization:** We utilized various visualization techniques to explore the data distributions and relationships between features. This helped in gaining insights into the dataset and detecting potential issues such as imbalance or outliers.
- **Building Features:** Feature engineering was applied to enhance the model's learning capability. We built features using different vectorization techniques such as:
 - **Count Vectorization**
 - **TF-IDF (Term Frequency-Inverse Document Frequency)**
 - **Word2Vec**
 - **GloVe (Global Vectors for Word Representation)**

These approaches allowed us to transform raw text data into meaningful numerical representations.

- **Model Selection:** We explored and implemented a variety of models, including:
 - **Graphical Models:** Bayesian Networks, Hidden Markov Models (HMM)
 - **Support Vector Machines (SVM):** with different kernels and regularization techniques
 - **PCA for Dimension Reduction:** with feature selection strategies, topic modeling, variance analysis, and chi-squared testing
 - **LDA as Linear Classifier**
 - **Ensemble Methods:** Random Forest, XGBoost, Stacking, and Voting strategies using Logistic Regressions and other selected models
 - **Discriminative Models:** Logistic Regression, BiLSTM-CRF (to leverage the superior performance of the BiLSTM architecture)

- **Training, Evaluating, and Testing:** We trained the models using appropriate training-validation splits and evaluated them using consistent metrics. Hyperparameter tuning and model validation helped ensure the robustness of our solutions.
- **Performance Comparison:** We performed a comprehensive comparison between the models, analyzing their strengths, weaknesses, and suitability for the given task. This analysis guided us in selecting the best-performing approach.

7.3 Special Thanks

We extend our sincere gratitude to our advisor, Dr. Nguyen An Khuong, for his invaluable guidance throughout our journey in machine learning and sentiment analysis. His mentorship has been instrumental in deepening our understanding of Machine Learning techniques, feature engineering, and model evaluation, enabling us to tackle the challenges of sentiment classification with confidence.

Beyond academic support, Dr. Nguyen An Khuong has provided insightful career advice and encouraged us to develop critical thinking and problem-solving skills in real-world machine learning applications. His encouragement has fostered an environment of continuous learning, inspiring us to explore innovative approaches in Machine Learning while maintaining a strong foundation in machine learning principles.

We are grateful for his dedication, which has not only enhanced our technical expertise but also prepared us for future academic and professional endeavors in the field of AI and Machine Learning.